

Computer Fundamentals with Minecraft Education

A K–5 Innovative Unit Design

[Candidate Name]

University of Rochester — Warner School of Education

EDU437: Computer Science Theory & Practice

Grade Bands: K–2 (Lower Elementary) and 3–5 (Upper Elementary)

Platform: Minecraft Education Edition + Microsoft MakeCode

Standards: NYS CS & Digital Fluency (2020) + CSTA K–12 CS Standards (2017)

Framework: Understanding by Design (UbD) / Backward Design

March 2026

Table of Contents

1	Unit Overview: UbD Stage 1 — Desired Results	5
1.1	Unit A: Computing with Minecraft (K–2)	5
1.2	Unit B: Coding FUNdamentals (3–5)	6
2	Unit A: Computing with Minecraft — Lesson Plans (K–2)	7
2.1	Lesson A-1: What Is a Computer? Hardware Explorers	7
2.1.1	Formative Assessment	8
2.1.2	Summative Assessment	8
2.2	Lesson A-2: My First Algorithm — The Sequencing Challenge	11
2.2.1	Formative Assessment	12
2.2.2	Summative Assessment	12
2.3	Lesson A-3: Looping the Agent — When We Repeat	14
2.3.1	Formative Assessment	15
2.3.2	Summative Assessment	15
2.4	Lesson A-4: Debugging — Be the Bug Fixer	17
2.4.1	Formative Assessment	17
2.4.2	Summative Assessment	18
2.5	Lesson A-5: Data All Around Us	20
2.5.1	Formative Assessment	21
2.5.2	Summative Assessment	21
2.6	Lesson A-6: Safe in the Online World — Digital Citizenship	23
2.6.1	Formative Assessment	24
2.6.2	Summative Assessment	24
2.7	Lesson A-7: Agency — Mission Planning	26
2.7.1	Formative Assessment	27
2.7.2	Summative Assessment	27
2.8	Lesson A-8: Creative Builders — Build with Code	29
2.8.1	Formative Assessment	30
2.8.2	Summative Assessment	30
2.9	Lesson A-9: Sorting and Patterns	32
2.9.1	Formative Assessment	33
2.9.2	Summative Assessment	33
2.10	Lesson A-10: Animal Habitat Challenge — Summative Assessment	35
2.10.1	Formative Assessment	36
2.10.2	Summative Assessment	36
3	Unit B: Coding FUNdamentals — Lesson Plans (3–5)	38
3.1	Block 1: Animal Conservation (Lessons B-1 through B-6)	38
3.2	Lesson B-1: Computational Thinking with Sea Turtles — Decomposition	38
3.2.1	Formative Assessment	39
3.2.2	Summative Assessment	39
3.3	Lesson B-2: Loops for Polar Bears	41
3.3.1	Formative Assessment	42
3.3.2	Summative Assessment	42
3.4	Lesson B-3: Debugging in the Wild — Panda Rescue	44
3.4.1	Formative Assessment	45
3.4.2	Summative Assessment	45
3.5	Lesson B-4: Pair Programming — Gray Wolf Collaboration	47

3.5.1	Formative Assessment	48
3.5.2	Summative Assessment	48
3.6	Lesson B-5: Variables — Animal Data Dashboard	50
3.6.1	Formative Assessment	51
3.6.2	Summative Assessment	51
3.7	Lesson B-6: Block 1 Summative — Animal Challenge Showcase	53
3.7.1	Formative Assessment	54
3.7.2	Summative Assessment	54
3.8	Block 2: Space Exploration (Lessons B-7 through B-12)	56
3.9	Lesson B-7: Conditionals — Moon Landing	56
3.9.1	Formative Assessment	57
3.9.2	Summative Assessment	57
3.10	Lesson B-8: Operators — Mercury Math	59
3.10.1	Formative Assessment	60
3.10.2	Summative Assessment	60
3.11	Lesson B-9: Events — Venus Launch Sequence	62
3.11.1	Formative Assessment	63
3.11.2	Summative Assessment	63
3.12	Lesson B-10: Loops + Conditionals — Mars Exploration	65
3.12.1	Formative Assessment	66
3.12.2	Summative Assessment	66
3.13	Lesson B-11: Networks — Jupiter Station	68
3.13.1	Formative Assessment	69
3.13.2	Summative Assessment	69
3.14	Lesson B-12: Neptune Expedition — Block 2 Summative	71
3.14.1	Formative Assessment	72
3.14.2	Summative Assessment	72
3.15	Block 3: Time Travel (Lessons B-13 through B-18)	74
3.16	Lesson B-13: Dynamic Variables — Dinosaur Age	74
3.16.1	Formative Assessment	75
3.16.2	Summative Assessment	75
3.17	Lesson B-14: Functions — Ancient Rome Aqueducts	77
3.17.1	Formative Assessment	78
3.17.2	Summative Assessment	78
3.18	Lesson B-15: Refactoring — Wild West Code Cleanup	80
3.18.1	Formative Assessment	81
3.18.2	Summative Assessment	81
3.19	Lesson B-16: Future City — Multiplayer Collaboration	83
3.19.1	Formative Assessment	84
3.19.2	Summative Assessment	84
3.20	Lesson B-17: CS in the Real World — Impacts of Computing	86
3.20.1	Formative Assessment	87
3.20.2	Summative Assessment	87
3.21	Lesson B-18: Grand Exhibition Capstone	89
3.21.1	Formative Assessment	90
3.21.2	Summative Assessment	90
4	Appendix A: Unit A Summative Rubric (Lesson A-10)	93
5	Appendix B: Unit B Summative Rubric (Lesson B-18)	94

6	Appendix C: Exit Ticket Templates	95
6.1	K–2 Exit Ticket Template	95
6.2	3–5 Exit Ticket Template	95
7	Appendix D: Vocabulary Reference Sheets	96
7.1	K–2 Vocabulary (with Minecraft Associations)	96
7.2	3–5 Vocabulary (with Code Examples)	96
8	Appendix E: Teacher Quick Reference — Minecraft Education Setup	98
8.0.1	Account Requirements	98
8.0.2	Sharing Worlds with Students	98
8.0.3	Device Capacity and Requirements	98
8.0.4	Common Issues and Fixes	98
8.0.5	Game Modes	99
9	References	100

Unit Overview: UbD Stage 1 — Desired Results

Unit A: Computing with Minecraft (K–2)

Title	Computing with Minecraft
Grade Band	K–2 (Ages 5–7)
Duration	10 lessons × 45 min ≈ 10 hours
Big Idea	Computers are tools that follow precise instructions. We can understand and direct how they work.
Essential Questions	What is a computer and how does it work? What is an algorithm and why does order matter? How do we find and fix mistakes in our thinking and our code? How can technology help us and others?
NYS CSDE Standards	K-1.CT.4, K-1.CT.6, K-1.CT.8, K-1.CT.9, K-1.CT.10, K-1.NSD.1, K-1.NSD.2, K-1.NSD.3, K-1.CY.1, K-1.CY.2, K-1.DL.2, K-1.DL.7, 2-3.CT.1, 2-3.CT.6, 2-3.CT.9, 2-3.NSD.2, 2-3.CY.2
CSTA Standards	1A-CS-01, 1A-CS-02, 1A-CS-03, 1A-AP-08, 1A-AP-10, 1A-AP-11, 1A-AP-14, 1A-NI-04, 1A-DA-05, 1A-DA-06, 1A-IC-17, 1A-IC-18
Summative Assessment	Animal Habitat Challenge (Lesson A-10): students plan + code a Minecraft mission, present to a partner, evaluated on 4-criteria rubric
Key Understandings	1. Algorithms are step-by-step instructions computers follow exactly. 2. Loops reduce repetition in code. 3. Bugs are expected; debugging is a skill. 4. Data can be collected, displayed, and used to answer questions. 5. We have responsibilities as digital citizens.

Unit B: Coding FUNdamentals (3–5)

Title	Coding FUNdamentals
Grade Band	3–5 (Ages 8–11)
Duration	18 lessons × 45 min ≈ 18 hours (3 thematic blocks)
Big Idea	Programming is a creative, iterative process of problem-solving. Complex programs are built from simple, reusable parts.
Essential Questions	How do we break complex problems into manageable steps? How do programs make decisions? What makes code efficient, readable, and maintainable? How does computing technology affect society?
NYS CSDF Standards	2-3.CT.1, 2-3.CT.4, 2-3.CT.6, 2-3.CT.7, 2-3.CT.8, 2-3.CT.9, 2-3.CT.10, 2-3.NSD.2, 2-3.NSD.4, 2-3.CY.2, 2-3.DL.2, 4-6.CT.1, 4-6.CT.4, 4-6.CT.6, 4-6.CT.7, 4-6.CT.8, 4-6.CT.9, 4-6.CT.10, 4-6.NSD.2, 4-6.NSD.4, 4-6.CY.1, 4-6.CY.2, 4-6.IC.1, 4-6.IC.6
CSTA Standards	1B-AP-08, 1B-AP-09, 1B-AP-10, 1B-AP-11, 1B-AP-15, 1B-AP-16, 1B-CS-01, 1B-CS-02, 1B-CS-03, 1B-NI-04, 1B-NI-05, 1B-DA-06, 1B-DA-07, 1B-IC-18, 1B-IC-19
Summative Assessment	Grand Exhibition Capstone (Lesson B-18): students design + present a complete Minecraft world demonstrating all 6 core CS concepts
Key Understandings	1. Decomposition makes complex problems solvable. 2. Conditionals and loops create flexible, adaptive programs. 3. Variables allow programs to store and change information. 4. Functions enable code reuse and clarity. 5. Testing and debugging are iterative, professional practices. 6. Computing technology shapes and is shaped by society.

Unit A: Computing with Minecraft — Lesson Plans (K–2)

Lesson A-1: What Is a Computer? Hardware Explorers

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	K–2
Subject Area	Computer Science / Digital Literacy
Title of Unit	Computing with Minecraft
Lesson Title	What Is a Computer? Hardware Explorers
Duration	45 minutes

1. Content Area

This lesson introduces the predominant content area of computer science, specifically computing systems and hardware. Students explore the physical components of a computer — keyboard, mouse, monitor, and internal parts such as the CPU and storage — and learn the foundational vocabulary of input, output, hardware, and software. Cross-curricular connections to English Language Arts are embedded as students add new technical vocabulary to personal word journals and practice using terms in complete sentences. The lesson also connects to science standards of observation and classification as students sort devices into categories.

2. Purposes/Goals

Students will develop a foundational understanding that computers are machines made of interconnected parts, each serving a specific function. The enduring Big Idea is: Computers are made of parts that work together to receive information, process it, and show results. This lesson matters because understanding hardware is the first step toward computational literacy — students must understand the physical tool before they can understand the instructions (software) that make it work. This lesson launches the unit’s investigation of how computers follow precise instructions.

3. Objectives

- Students will be able to identify at least three hardware components of a computer (e.g., keyboard, monitor, mouse, CPU, storage) and describe the function of each.
- Students will be able to match physical hardware to their in-game Minecraft analogs (chest = storage, furnace = CPU, crafting table = input device).
- Students will be able to use the terms input, output, hardware, and software correctly when describing how a computer works.
- Students will be able to distinguish between hardware (physical parts) and software (instructions/programs) with at least 80% accuracy on the exit ticket.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF Standard K-1.NSD.1 (Identify the basic components of a computing device, describing what each component does), demonstrated when students examine real hardware and label device diagrams. It also addresses K-1.NSD.2 (Understand and describe how humans interact with computing devices), practiced as students navigate the Minecraft world using keyboard, mouse, and touchscreen inputs. The lesson targets K-1.CT.6 (Follow an algorithm to complete a task), introduced as students follow a one-step MakeCode instruction. CSTA 1A-CS-01 (Select and operate appropriate software) is addressed as students interact with Minecraft. CSTA 1A-CS-02 (Use appropriate terminology in identifying and describing the function of common physical components) is the central skill practiced throughout.

5. Assessment

Formative Assessment

The teacher circulates during the Minecraft exploration and uses an observation checklist to note whether each student interacts with at least two NPCs and can verbally name the hardware analog. During MakeCode, the teacher captures a screenshot of each student's single-block program. A quick-check question at the 25-minute mark asks partners to name one input and one output device. The exit ticket (label a blank device diagram) provides written formative data. For SWD, a multiple-choice matching format is available. For ELL students, the exit ticket includes vocabulary images and home-language labels.

Summative Assessment

This lesson is the foundation for the Unit A summative (Lesson A-10: Animal Habitat Challenge). Hardware vocabulary and the hardware/software distinction will be assessed in the rubric's Communication criterion. SWD have modified rubric expectations documented in their learning profiles; ELL students may use visual aids and bilingual glossaries during the summative.

6. Community Knowledge and Experience

Many students arrive with rich prior experiences using technology at home. The lesson opens by asking, 'What devices do you use at home? What do you do with them?' This validates out-of-school knowledge and positions every child as someone who already knows about computers. ELL students are included with vocabulary cards featuring images and home-language translations. Immersive Reader is activated for all NPC text. Students with disabilities participate through multi-modal entry points: visual, auditory, and kinesthetic. The teacher models curiosity: 'When I first learned about CPUs, I was amazed — let's see what you think!'

7. Procedures

Time	Teacher will:	Student will:
0–5 min	Hold up a real keyboard. Ask: 'What is this? What does it do?' Chart responses under 'Things We Know About Computers.' Affirm all responses including those in home languages.	Share prior knowledge about keyboards and computers. Turn and talk with a partner.
5–10 min	Project a labeled computer diagram. Introduce vocabulary: input, output, CPU, storage. Add each word to the class word wall.	Listen, repeat vocabulary aloud (choral response), add words to personal vocabulary journals with drawings.
10–15 min	Distribute 'Hardware Scavenger Hunt' worksheets. Students find 5 input/output devices in the classroom.	Walk around the room identifying devices (document camera, projector, headphones, mouse). Record findings on worksheet.
15–20 min	Open Minecraft Education Creative Mode 'Hardware Museum' world. Explain: 'The chest is like storage. The furnace is like the CPU.'	Watch demonstration, ask questions, prepare to log in.
20–30 min	Support students exploring the Hardware Museum. Each NPC represents a component. Circulate asking: 'Which computer part is this NPC?'	Navigate Minecraft world. Interact with NPCs using Immersive Reader. Identify hardware components.
30–35 min	Open MakeCode. Demo one 'move forward' block: 'This is software — an instruction.' Students drag one block and run it.	Open MakeCode. Drag one 'move forward' block. Run the program. Observe the Agent move.
35–40 min	Pause coding. 'The block is software. The keyboard is hardware. Tell your partner the difference.' Listen to conversations.	Turn and talk. Explain: 'Hardware is something I can touch; software is the instruction.'
40–45 min	Distribute exit ticket: label a blank device diagram. Collect for formative data.	Label hardware components on diagram. Turn in ticket.

Backup Plan: If devices are unavailable, use printed hardware diagram cards and a physical sorting activity. Students sort laminated cards showing keyboards, monitors, mice, CPUs, and storage into 'Input' and 'Output' categories.

8. Differentiated Instruction

UDL Multiple Means of Representation: content presented visually (diagrams, Minecraft), auditorily (teacher narration, Immersive Reader), and kinesthetically (scavenger hunt). Multiple Means of Action and Expression: verbal turn-and-talk, written exit tickets, and in-game interaction. Multiple Means of Engagement: Minecraft provides motivating context; scavenger hunt offers movement. IEP accommodations: extended time, modified exit ticket with word bank, preferential seating, 1:1 check-ins. ELL modifications: bilingual vocabulary cards, Immersive Reader, sentence frames, bilingual peer pairing. Extension: photograph a device at home and annotate hardware components.

9. Resources

Materials: real keyboard, projector, Minecraft Education, pre-built 'Hardware Museum' world, MakeCode editor, scavenger hunt worksheet, exit ticket diagrams, vocabulary flashcards, vocabulary journals, word wall cards. Assistive technology: Minecraft's built-in Immersive Reader for text-to-speech and translation. Resources distributed at lesson start; devices pre-loaded by aide.

10. Applications, Connections & Extensions

Lesson A-2 extends understanding of software with multi-step programs. Lesson A-6 revisits hardware when discussing internet connections. Real-world connections: identifying hardware in smartphones, ATMs, and self-checkout kiosks. Extension: 'Design Your Own Computer' drawing activity; family interview about first computers.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson A-2: My First Algorithm — The Sequencing Challenge

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	K–2
Subject Area	Computer Science / Digital Literacy
Title of Unit	Computing with Minecraft
Lesson Title	My First Algorithm — The Sequencing Challenge
Duration	45 minutes

1. Content Area

This lesson focuses on algorithms and sequencing — the core building blocks of programming. Students learn that an algorithm is a precise, ordered set of steps and that the order of steps determines the outcome. Cross-curricular connections include mathematics (sequential ordering, positional language) and ELA (narrative sequencing, transition words).

2. Purposes/Goals

Students will understand that all computer programs are built from algorithms. The enduring Big Idea is: An algorithm is an ordered set of steps; the order matters because changing the order changes the result. Sequencing is the foundational programming concept upon which loops, conditionals, and all higher-order structures depend.

3. Objectives

- Students will be able to define 'algorithm' in their own words as a set of steps in a specific order.
- Students will be able to create a 4+ step MakeCode sequence to navigate the Agent through a maze.
- Students will be able to record their plan on paper before coding, practicing decomposition.
- Students will be able to explain why the order of steps matters by describing what happens when steps are rearranged.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF Standard K-1.CT.4 (Identify and solve a simple problem through the construction of a solution), as students plan and code a maze solution. K-1.CT.6 (Follow an algorithm to complete a task) is the core standard. K-1.CT.10 (Create a plan and follow a set of steps) is addressed during pre-coding planning. CSTA 1A-AP-08 (Model daily processes by creating and following algorithms) is addressed during the unplugged 'Robot Simon Says' activity. CSTA 1A-AP-10 (Develop programs with sequences and simple loops) is practiced in MakeCode. CSTA 1A-AP-11 (Decompose the steps needed to solve a problem into a precise sequence of instructions) corresponds to the planning-before-coding approach.

5. Assessment

Formative Assessment

The teacher observes whether students can follow multi-step physical instructions during the unplugged activity. Planning grids are reviewed before coding begins. MakeCode screenshots are captured upon maze completion. Quick-check at 30 min: 'Did your Agent follow your plan?' The exit ticket (4-step PBJ sandwich algorithm) is scored for correct sequencing. SWD receive cut-and-paste version; ELL students get sentence frames.

Summative Assessment

This lesson establishes the Algorithm Design criterion for the Unit A summative rubric. Students' plan-to-code workflow is the foundation of that criterion. SWD may use visual planning templates; ELL students may annotate plans with illustrations.

6. Community Knowledge and Experience

Students connect algorithms to sequential tasks from daily life — getting dressed, brushing teeth. They share morning routines, validating every child's experience as algorithmic thinking. ELL students share sequences from home cultures. Students with disabilities participate in the unplugged activity through adapted motions. The teacher connects: 'When I cook dinner, I follow an algorithm!'

7. Procedures

Time	Teacher will:	Student will:
0–3 min	Gather on rug. Introduce 'algorithm.' 'Say it with me: AL-go-rith-m. An algorithm is a set of steps in a specific order.'	Repeat 'algorithm' aloud. Listen to the introduction.
3–8 min	Play 'Robot Simon Says' with multi-step instructions. Ask: 'What happens if I change the order?'	Follow physical instructions. Discuss how changing order changes outcomes.
8–15 min	Project a Minecraft maze. Model drawing arrows on a planning grid. Think aloud: 'I need to plan BEFORE I code.'	Watch. Receive planning grid. Begin drawing planned sequence using directional arrows.
15–20 min	Check planning grids at tables. Approve 4+ step plans. Ask guiding questions for struggling students.	Complete planning grids. Get teacher approval.
20–30 min	Students load Coding Fundamentals 1 Basic Moves world. 'Translate your paper plan into MakeCode blocks.' Circulate.	Open Minecraft. Drag movement blocks to match paper plan. Run and test.
30–38 min	Facilitate class discussion: 'Who had to change something? That's normal!' Chart responses.	Share what worked and what didn't. Listen to peers.
38–45 min	Distribute exit ticket: draw a 4-step PBJ sandwich algorithm. Collect.	Complete and submit exit ticket.

Backup Plan: If Minecraft is unavailable, use Code.org offline 'Move It, Move It' card activity — students program a peer to navigate a floor grid using directional cards.

8. Differentiated Instruction

UDL Representation: algorithm introduced verbally, kinesthetically, and visually. Action/Expression: physical movement, paper planning, coding, verbal discussion. Engagement: goal-oriented Minecraft maze. IEP: pre-drawn sequence strips, extended time, reduced maze complexity. ELL: bilingual directional cards, sentence frames, partner support. Extension: design a maze for a partner to solve.

9. Resources

Materials: projector, Minecraft Education, Coding Fundamentals 1 Basic Moves world, MakeCode, planning grid worksheets, exit ticket worksheets, directional vocabulary cards. Assistive technology: Immersive Reader, screen magnification, adaptive keyboard/mouse.

10. Applications, Connections & Extensions

Lesson A-3 builds on this by introducing loops for repetitive sequences. Real-world connections: algorithms in traffic lights, elevators, vending machines. Extension: 'Family Algorithm' take-home challenge.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson A-3: Looping the Agent — When We Repeat

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	K–2
Subject Area	Computer Science / Digital Literacy
Title of Unit	Computing with Minecraft
Lesson Title	Looping the Agent — When We Repeat
Duration	45 minutes

1. Content Area

This lesson addresses iteration (loops). Students learn that a loop repeats instructions a specified number of times, making code shorter and more efficient. Cross-curricular connections include mathematics (counting, multiplication as repeated addition) and music (rhythmic patterns).

2. Purposes/Goals

Students will discover that loops eliminate redundant code. The Big Idea is: A loop tells the computer to repeat steps; loops make programs shorter and more powerful. Loops are a fundamental programming construct; understanding repetition here prepares students for nested loops and conditionals later.

3. Objectives

- Students will be able to explain why loops reduce code length and increase efficiency.
- Students will be able to use a 'repeat' block in MakeCode to build a fence in Minecraft.
- Students will be able to identify when a loop produces the wrong outcome and revise the loop count or body.
- Students will be able to compare code with and without loops and articulate which is better and why.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF K-1.CT.8 (Identify a task that is repetitive and would benefit from automation), demonstrated when students recognize that individual commands are tedious. K-1.CT.10 (Create a plan and follow steps) continues from A-2. CSTA 1A-AP-10 (Develop programs with sequences and simple loops) is the primary standard. CSTA 1A-AP-11 (Decompose steps into a precise sequence) is reinforced during loop identification.

5. Assessment

Formative Assessment

Teacher observes clapping warm-up for accuracy. During MakeCode comparison, asks: 'How many blocks without the loop? With?' Screenshots captured during fence challenge. Share-out at 40 min is a verbal check. ELL students explain using gestures. SWD receive 5-minute check-ins.

Summative Assessment

Loops are assessed in the Unit A summative rubric under 'Use of MakeCode Blocks' — students must use both sequence and loop to meet 'Meets.' This lesson teaches that skill.

6. Community Knowledge and Experience

Loops connect to singing choruses, jumping rope, and bouncing balls. The clapping warm-up draws from musical traditions across cultures. ELL students share repetitive patterns from home-language songs. SWD participate through adapted movements.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	Lead clapping pattern: 'Clap, stomp' × 3. Write it as 6 lines. Then: 'Repeat 3 times: clap, stomp.' Introduce 'loop.'	Follow the pattern. Observe shorthand. Say 'loop' aloud.
5–15 min	Open MakeCode. Show 4 individual 'move forward' blocks vs. one 'repeat 4' with 'move forward.' Run both. 'Which is shorter?'	Watch both versions. Compare. Predict what happens with repeat 10.
15–30 min	Load 'Fence Garden Challenge' world. 'Use a repeat loop to place fence posts.' Circulate checking for loop usage.	Write MakeCode program using repeat loop. Test. Adjust loop count as needed.
30–40 min	Extension: 'Can you build a 10-block wall with only 3 lines of code?' Introduce nested loop preview for fast finishers.	Attempt extension challenge. Experiment with loop counts.
40–45 min	2 students display screens and explain their loops. Celebrate.	Present loop code. Add 'loop' to vocabulary journal.

Backup Plan: Loop Relay Race: students act out a loop physically (run to cone and return × N), then record 'repeat N' on paper. Paper-based coding with repeat block diagrams.

8. Differentiated Instruction

UDL Representation: body movement, visual code comparison, hands-on coding. Action/Expression: coding, verbal, or drawing a loop diagram. Engagement: clear visible goal with immediate feedback. IEP: reduced fence length, visual step card, 1:1 support. ELL: visual loop diagram, bilingual card, partner support. Extension: nested loops; 'loop story.'

9. Resources

Materials: projector, Minecraft Education, MakeCode, 'Fence Garden Challenge' world, vocabulary journals, word wall card for 'loop.' *Assistive technology:* Immersive Reader, adaptive inputs.

10. Applications, Connections & Extensions

Lesson A-4 (Debugging) asks students to find errors in loop-based code. *Real-world:* washing machines, traffic lights, music choruses. *Extension:* 'Loop Hunt' homework — find 3 loops in daily life.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson A-4: Debugging — Be the Bug Fixer

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	K–2
Subject Area	Computer Science / Digital Literacy
Title of Unit	Computing with Minecraft
Lesson Title	Debugging — Be the Bug Fixer
Duration	45 minutes

1. Content Area

This lesson focuses on debugging — identifying, analyzing, and correcting errors in programs. Cross-curricular connections include ELA (reading comprehension: predicting and checking) and social-emotional learning (perseverance, growth mindset).

2. Purposes/Goals

Students will internalize that errors are normal and fixable. The Big Idea is: Bugs are errors; every programmer encounters bugs. Debugging is a professional skill, not a sign of failure. Fear of mistakes is the biggest barrier to computational thinking for young learners; this lesson normalizes the debugging process with a systematic five-step strategy.

3. Objectives

- Students will be able to define 'bug' and 'debug' accurately.
- Students will be able to follow the 5-step debugging process: Read, Predict, Run, Compare, Fix.
- Students will be able to identify and fix at least one error in a pre-written MakeCode program.
- Students will be able to describe the bug and their fix in writing or drawing.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF K-1.CT.9 (Test and debug a simple program to ensure it works as intended), the central activity. CSTA 1A-AP-14 (Debug errors in an algorithm or program that includes sequences and simple loops) directly corresponds. CSTA 1A-CS-03 (Describe basic hardware and software problems) is practiced as students articulate errors.

5. Assessment

Formative Assessment

During the story, teacher checks: 'What was the bug in the recipe?' During debugging, a checklist tracks whether each student can: run, identify, locate, and fix the error. The written/drawn reflection is the primary artifact. SWD may record verbally or draw. ELL students use sentence frames.

Summative Assessment

Debugging is assessed in the summative rubric under 'Debugging Evidence.' This lesson teaches the systematic approach that criterion evaluates.

6. Community Knowledge and Experience

The recipe analogy connects to home cooking experiences. Students share mistakes they've made while building or cooking — normalizing error. Growth-mindset framing supports SWD who may have internalized failure narratives. ELL students benefit from the visual nature of debugging.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	Read 'recipe with a bug': '3 cups of SALT' instead of peanut butter. 'Did you catch the bug?' Connect to programming bugs.	Identify the error. Laugh. Connect 'bug' to an error in instructions.
5–8 min	Post 5-Step Debugging Process. Walk through with the recipe example.	Read steps aloud. Practice with the recipe. Copy steps onto reference card.
8–20 min	Load pre-broken MakeCode program on projector (Agent turns wrong). Walk through 5 steps with the class. Students identify and fix the error.	Follow along. Predict, observe, identify the turn-left block as the error. Fix to turn-right.
20–35 min	Distribute 3 pre-broken programs. Each has a different bug type: wrong direction, wrong loop count, missing block. Students debug all three.	Open each buggy program. Follow 5-step process. Fix bugs and test.
35–45 min	2–3 students share. Distribute reflection: 'The bug was _____. I fixed it by _____.' Close: 'You are official Bug Fixers!'	Share experiences. Complete reflection. Celebrate.

Backup Plan: Paper-based debugging: printed algorithm cards with intentional errors; students cross out and rewrite using the 5-step process.

8. Differentiated Instruction

UDL Representation: story, visual chart, hands-on coding. Action/Expression: coding, verbal, drawing, or voice memo. Engagement: 'bug fixer' identity motivates persistence. IEP: 1–2 programs instead of 3, visual debugging card, 1:1 support. ELL: sentence frames, bilingual vocabulary, Immersive Reader. Extension: create a buggy program for a partner to debug.

9. Resources

Materials: projector, Minecraft Education, 3 pre-broken MakeCode programs, 5-Step Debugging poster, reference cards, reflection worksheets, vocabulary cards. Assistive technology: Immersive Reader, text-to-speech.

10. Applications, Connections & Extensions

Lesson A-5 introduces data collection; debugging skills transfer when data programs fail. Real-world: NASA debugging teams. Extension: 'Bug Journal' for the rest of the unit.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson A-5: Data All Around Us

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	K–2
Subject Area	Computer Science / Digital Literacy
Title of Unit	Computing with Minecraft
Lesson Title	Data All Around Us
Duration	45 minutes

1. Content Area

This lesson introduces data — what it is, how we collect it, and how we organize it to find patterns. Cross-curricular connections include mathematics (graphing, counting) and science (observation, evidence-based claims).

2. Purposes/Goals

Students will discover that data is information we can collect, organize, and interpret. The Big Idea is: Data is information we collect; organizing data as charts and graphs helps us find patterns and make predictions. Data literacy is essential in an increasingly data-driven world.

3. Objectives

- Students will be able to define 'data' as information that can be collected and organized.
- Students will be able to collect data by counting blocks in Minecraft and recording counts.
- Students will be able to create a bar chart from their collected data.
- Students will be able to write a one-sentence data claim based on their chart.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF K-1.CT.2 (Identify a simple problem addressable with data) and K-1.CT.3 (Collect and organize data). CSTA 1A-DA-05 (Store, copy, search, retrieve, modify, and delete data) is introduced through Minecraft inventory. CSTA 1A-DA-06 (Collect and organize data in a chart or graph) is directly practiced. CSTA 1A-DA-07 (Identify patterns in data) corresponds to the analysis discussion.

5. Assessment

Formative Assessment

Teacher spot-checks counting accuracy, verifies bar charts have labeled axes, and scores the data claim for accuracy and connection to the chart. SWD may dictate claims. ELL students use sentence frames.

Summative Assessment

Data skills contribute to the summative where students analyze animal habitat data. This lesson provides foundational data literacy.

6. Community Knowledge and Experience

The class survey about favorite mobs generates excitement and positions every student as a data contributor. ELL students point to images. SWD use adapted counting methods. Teacher connects: 'Data is everywhere!'

7. Procedures

Time	Teacher will:	Student will:
0–8 min	Class survey: 'What is your favorite Minecraft mob?' Tally on the board. Introduce 'data.'	Vote by raising hand. Identify the most popular. Repeat 'data.'
8–15 min	Open 'Data Garden' Minecraft world. Model counting blocks in Zone 1 and recording tallies.	Watch teacher model. Open Minecraft. Navigate to assigned zone.
15–25 min	Students count 3 block types in their zone. Circulate and support accurate counting.	Count and record using tally marks. Compare with neighbors.
25–35 min	Distribute graph paper. Model drawing a bar chart. Assist with labels and bar heights.	Create bar chart from tally data. Label axes. Color bars.
35–42 min	Discussion: 'Which block is most common? How do you know?' Students write a data claim.	Analyze chart. Write: 'The most common block is ____ because ____.'
42–45 min	2 students share claims. Discuss why zones may differ. Close: 'You are data scientists!'	Listen to peers. Discuss variation. Add 'data' to journal.

Backup Plan: Physical tally with manipulatives: bags of colored blocks to sort, count, and graph on paper.

8. Differentiated Instruction

UDL Representation: kinesthetic survey, visual tallies, Minecraft counting. Action/Expression: tallying, graphing, writing, verbal. Engagement: authentic Minecraft context. IEP: pre-labeled graph paper, reduced counting area, 1:1 support. ELL: visual vocabulary, sentence frames. Extension: compare two zones' data; predict a fourth zone.

9. Resources

Materials: projector, Minecraft Education, 'Data Garden' world, data collection worksheets, graph paper, colored pencils, vocabulary cards. Assistive technology: Immersive Reader, adaptive inputs.

10. Applications, Connections & Extensions

Data returns in A-9 (Sorting and Patterns). Real-world: weather forecasts, sports statistics. Extension: 'Data Detective' homework.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson A-6: Safe in the Online World — Digital Citizenship

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	K–2
Subject Area	Computer Science / Digital Literacy
Title of Unit	Computing with Minecraft
Lesson Title	Safe in the Online World — Digital Citizenship
Duration	45 minutes

1. Content Area

This lesson addresses cybersecurity and digital citizenship. Students learn foundational online safety rules including password privacy, personal information protection, and responsible digital behavior. Cross-curricular: social studies (community rules) and ELA (reading informational text).

2. Purposes/Goals

Students will understand that digital communities have safety rules just like physical ones. The Big Idea is: Just like we have rules to stay safe in real life, we have rules to stay safe online. As young children increasingly use technology, developing digital citizenship habits early is essential.

3. Objectives

- Students will be able to identify at least three types of personal information to keep private online.
- Students will be able to explain why passwords should not be shared except with a trusted adult.
- Students will be able to describe one safe action to take if someone online asks for personal information.
- Students will be able to create a 'Safety Pledge' card with one digital safety rule.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF K-1.CY.1 (Identify personal information not to share online), K-1.CY.2 (Identify and use safe internet behaviors), and K-1.DL.7 (Communicate respectfully online). CSTA 1A-NI-04 (Explain passwords and their purpose), 1A-IC-17 (Work respectfully online), and 1A-IC-18 (Keep login information private) are all addressed.

5. Assessment

Formative Assessment

Teacher observes NPC puzzle completion and asks comprehension questions. Safety Pledge card is the primary artifact. SWD may draw or dictate. ELL students use bilingual templates.

Summative Assessment

Digital citizenship is assessed through responsible technology use during the summative. The rubric's Communication criterion includes respectful partner interaction.

6. Community Knowledge and Experience

Draws on students' existing knowledge of community rules. Students share family technology rules. ELL students participate through visual sorting activities. SWD benefit from multi-modal Minecraft environment.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	'What rules keep us safe in our classroom? At home?' Chart responses. 'Today we learn internet safety rules.'	Share safety rules. Listen to the internet connection.
5–10 min	Load 'Home Sweet Mine' Internet Safety world from the Library. Preview on projector.	Watch preview. Prepare to log in.
10–30 min	Students explore the world with NPC-guided puzzles about passwords, personal information, and safe choices. Circulate asking: 'What did the NPC teach you?'	Navigate world. Complete puzzles. Discuss learning with nearby peers.
30–40 min	Rug discussion: 'What if someone asks for your password? Your address?' Role-play scenarios.	Share strategies. Volunteer for role-play. Practice saying 'No' and telling a trusted adult.
40–45 min	Distribute Safety Pledge cards. Model: 'I will not share my password.' Display on bulletin board.	Create Safety Pledge with one rule. Sign it. Share with a partner.

Backup Plan: Card-sorting activity: sort laminated cards of personal information into 'Safe to Share' and 'Not Safe' piles. Discussion and pledge remain the same.

8. Differentiated Instruction

UDL Representation: NPCs, narration, role-play, discussion. Action/Expression: Minecraft interactions, verbal, role-play, written/drawn pledge. Engagement: immersive world, personal ownership through pledge. IEP: fill-in-the-blank pledge, 1:1 support. ELL: bilingual cards, Immersive Reader, picture-based sorting. Extension: 'Digital Safety Tips' poster for the hallway.

9. Resources

Materials: projector, Minecraft Education, 'Home Sweet Mine' world, Safety Pledge templates, crayons/markers, bulletin board. Assistive technology: Immersive Reader.

10. Applications, Connections & Extensions

Digital citizenship reinforced through responsible multiplayer use. Real-world: website passwords, privacy settings. Extension: 'Safety Superhero' comic strip take-home.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson A-7: Agency — Mission Planning

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	K–2
Subject Area	Computer Science / Digital Literacy
Title of Unit	Computing with Minecraft
Lesson Title	Agency — Mission Planning
Duration	45 minutes

1. Content Area

This lesson develops students’ agency and metacognitive skills by focusing on goal setting, planning, and self-directed problem-solving with the Minecraft Agent. Cross-curricular: ELA (writing goals/plans) and social-emotional learning (self-regulation, persistence).

2. Purposes/Goals

Students will take ownership of their learning by planning and executing a self-directed Agent mission. The Big Idea is: We can set goals, make plans, and direct the computer Agent to carry them out. This lesson transitions students from following teacher tasks to designing their own, building executive function skills essential for the summative.

3. Objectives

- Students will be able to set a clear, achievable goal for an Agent mission.
- Students will be able to write a step-by-step plan using at least four MakeCode blocks before coding.
- Students will be able to execute, test, and revise their plan at least once.
- Students will be able to reflect on whether their goal was achieved and what they would change.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF K-1.CT.4 (Identify and solve a problem), K-1.CT.6 (Follow an algorithm), and K-1.CT.10 (Create a plan and follow steps). CSTA 1A-AP-08 (Model daily processes with algorithms) and 1A-AP-11 (Decompose steps into precise instructions) are both addressed.

5. Assessment

Formative Assessment

Teacher reviews mission plans before coding, captures MakeCode screenshots, and asks: 'Did your Agent accomplish the mission?' Reflection worksheet is the primary artifact. SWD use planning templates. ELL plans may include drawings.

Summative Assessment

Directly prepares for the summative by practicing the plan-then-code workflow that the rubric's Algorithm Design criterion evaluates.

6. Community Knowledge and Experience

Open-ended mission design allows personal connections — students draw on interests and cultural knowledge. ELL students describe missions visually. SWD choose appropriate complexity.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	'Today, YOU decide the mission. You are the mission commander.' Show 3 example ideas on the board.	Listen. Begin thinking about their mission. Share ideas with a partner.
5–15 min	Distribute planning worksheets (My Goal, My Steps 1–6, Blocks I'll Use). Model completing one.	Complete worksheet with goal and 4+ steps. Identify needed blocks.
15–30 min	'Follow your plan. Debug if needed!' Circulate providing targeted support.	Translate plan into MakeCode blocks. Run and test. Debug errors.
30–40 min	'Make at least one revision to improve your mission.'	Review, revise, and retest their program.
40–45 min	Distribute reflection card. 1–2 students share. Close: 'YOU can plan and command the Agent!'	Complete reflection. Listen to peer shares.

Backup Plan: Paper-based: complete planning activity, then partner up to 'play computer' — one reads aloud while the other acts as the Agent.

8. Differentiated Instruction

UDL Representation: visual examples, oral modeling, peer discussion. Action/Expression: writing, drawing, coding, verbal. Engagement: open-ended choice maximizes autonomy. IEP: simplified template, 3 steps, pre-selected option, 1:1 support. ELL: picture-based worksheet, bilingual vocabulary, partner support. Extension: two-part mission; class presentation.

9. Resources

Materials: projector, Minecraft Education, MakeCode, planning worksheets, reflection cards, word wall cards. Assistive technology: Immersive Reader, adaptive inputs.

10. Applications, Connections & Extensions

Lesson A-8 extends to blueprint-based building. Real-world: Mars rover mission planning. Extension: 'Mission at Home' plan for a household chore.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson A-8: Creative Builders — Build with Code

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	K–2
Subject Area	Computer Science / Digital Literacy
Title of Unit	Computing with Minecraft
Lesson Title	Creative Builders — Build with Code
Duration	45 minutes

1. Content Area

This lesson synthesizes sequence and loop skills in a creative building context. Students design blueprints on paper and build them with MakeCode. Cross-curricular: mathematics (geometry, spatial reasoning) and art (design, aesthetics).

2. Purposes/Goals

Students apply sequences and loops together in an open-ended building project. The Big Idea is: We combine sequences and loops to create complex structures efficiently. This bridges isolated skill practice and the integrated application required for the summative.

3. Objectives

- Students will be able to design a blueprint on graph paper with dimensions and block types.
- Students will be able to write a MakeCode program combining sequences and loops for their structure.
- Students will be able to explain design choices and code to a partner using CS vocabulary.
- Students will be able to test, identify an improvement, and revise.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF K-1.CT.8 (Identify repetitive tasks for automation) and K-1.CT.10 (Create a plan and follow steps). CSTA 1A-AP-10 (Develop programs with sequences and loops) and 1A-AP-11 (Decompose steps) are practiced.

5. Assessment

Formative Assessment

Teacher reviews blueprints for completeness. During coding, checks for both sequence and loop blocks. Partner explanation assessed via observation rubric. SWD may demonstrate by pointing. ELL students use vocabulary reference cards.

Summative Assessment

Most direct preparation for the summative, which requires planning and coding a complete project. The blueprint-to-code workflow mirrors the rubric's Algorithm Design criterion.

6. Community Knowledge and Experience

Creative building connects to LEGO, block play, and community structures. Students draw inspiration from their neighborhoods. ELL students excel in visual/spatial tasks. SWD have accessible building options.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	Show 3 structures on projector. 'What patterns do you see? Where is repetition?' Circle repeated elements. 'Today you're architects.'	Observe structures. Identify repeated patterns.
5–15 min	Distribute graph paper. Model creating a blueprint. Guide loop identification.	Design blueprint. Label dimensions and block types. Identify loops.
15–30 min	'Translate your blueprint into code.' Circulate asking: 'Show me your loop in the blueprint.'	Code MakeCode program matching blueprint. Test.
30–35 min	'Find one thing to improve. Revise.'	Review, revise, and retest.
35–42 min	Pair students. 'Explain your code to your partner.'	Present blueprint and build. Explain using CS vocabulary.
42–45 min	Celebrate builds. Preview summative prep.	Celebrate. Add 'blueprint' to vocabulary journal.

Backup Plan: Paper blueprints plus paper 'code cards' (sequence and loop cards) arranged on desk. Partners predict what the build would look like.

8. Differentiated Instruction

UDL Representation: visual, spatial, digital. Action/Expression: drawing, coding, verbal. Engagement: creative choice, personal relevance. IEP: pre-started blueprint, simplified structure, 1:1 support. ELL: visual vocabulary, Immersive Reader, sentence frames. Extension: multi-room structure with nested loops; 'build manual' for a classmate.

9. Resources

Materials: projector, Minecraft Education, MakeCode, graph paper, colored pencils, sentence frames, vocabulary cards. Assistive technology: Immersive Reader, adapted materials.

10. Applications, Connections & Extensions

Lesson A-9 returns to data skills. Real-world: architects use CAD software. Extension: build a famous building and present research.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson A-9: Sorting and Patterns

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	K–2
Subject Area	Computer Science / Digital Literacy
Title of Unit	Computing with Minecraft
Lesson Title	Sorting and Patterns
Duration	45 minutes

1. Content Area

This lesson deepens data literacy through sorting, classifying, and pattern identification. Students organize Minecraft items by attributes and create pictographs. Cross-curricular: mathematics (sorting, classifying, graphing) and science (patterns, classification).

2. Purposes/Goals

Students practice organizing data to find patterns. The Big Idea is: Sorting and classifying data helps us see patterns and make predictions. Pattern recognition is a pillar of computational thinking essential for the summative and beyond.

3. Objectives

- Students will be able to sort Minecraft items by at least two attributes.
- Students will be able to create a pictograph from their sorting data.
- Students will be able to identify at least one pattern in organized data.
- Students will be able to make a prediction based on their observed pattern.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 2-3.CT.1 (Identify a problem solvable with a computational artifact, pre-viewing upper-grade standards) and K-1.CT.3 (Collect and organize data). CSTA 1A-DA-05 (Store and retrieve data) and 1A-DA-06 (Collect data in a chart) are addressed.

5. Assessment

Formative Assessment

Teacher checks sorting accuracy and pictograph quality. Pattern statements and predictions are collected. SWD sort physically; pictographs may use stamps. ELL students get image-based category labels.

Summative Assessment

Pattern recognition and data skills contribute to the summative where students analyze habitat data. Builds data literacy for the Algorithm Design criterion.

6. Community Knowledge and Experience

Sorting connects to everyday organizing — toys, laundry, books. Students share home practices. ELL students demonstrate sorting physically. SWD have manipulatives.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	Place mixed colored blocks on tables. 'Sort into groups — you choose the rule.' 3 minutes. 'What rule did you use?'	Sort physical blocks. Choose a rule. Explain to teacher or partner.
5–12 min	Connect to Minecraft inventory sorting. Model: natural vs. crafted items.	Watch. Open Minecraft. Practice sorting in chests.
12–25 min	In 'Sorting Station' world, students sort items from treasure chests. 'How are you grouping? Try a different way.'	Navigate and sort. Try two different attributes.
25–35 min	Distribute pictograph templates. Model drawing consistent symbols.	Create pictograph from sorting data. Label categories.
35–42 min	'What pattern do you see? Can you predict what's in the next chest?'	Write pattern sentence and prediction.
42–45 min	Share 2–3 pictographs. 'Next class is the big project!'	Listen. Prepare for summative.

Backup Plan: Physical sorting with buttons, beads, LEGO. Paper-based pictographs. Pattern identification and predictions remain.

8. Differentiated Instruction

UDL Representation: physical, digital, visual. Action/Expression: sorting, graphing, writing, discussing. Engagement: treasure chest discovery, open-ended rules. IEP: pre-sorted example, fewer categories, stamps. ELL: visual labels, bilingual vocabulary. Extension: double pictograph; sorting algorithm in pseudocode.

9. Resources

Materials: colored blocks, projector, Minecraft Education, 'Sorting Station' world, pictograph templates, crayons, vocabulary cards. Assistive technology: Immersive Reader, adapted materials.

10. Applications, Connections & Extensions

Lesson A-10 is the summative. Real-world: computers sort files, emails, search results. Extension: 'Sort the Library' — propose a system for classroom books.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson A-10: Animal Habitat Challenge — Summative Assessment

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	K–2
Subject Area	Computer Science / Digital Literacy
Title of Unit	Computing with Minecraft
Lesson Title	Animal Habitat Challenge — Summative Assessment
Duration	45 minutes

1. Content Area

This summative lesson integrates all K–2 CS skills: hardware, algorithms, sequences, loops, debugging, data, digital citizenship, planning, and creative building. Students design and code an animal habitat in Minecraft. Cross-curricular: science (animal habitats), ELA (presentation), mathematics (data collection).

2. Purposes/Goals

Students demonstrate mastery by independently planning, coding, debugging, and presenting a complete project. The Big Idea is: We can plan, code, debug, and present a complete project using all the computer science skills we have learned. This authentic assessment celebrates student growth and practices 21st-century skills.

3. Objectives

- Students will be able to write a 5+ step plan for building an animal habitat.
- Students will be able to code a working MakeCode program using sequences, loops, and at least one additional block type.
- Students will be able to identify and fix at least one bug, documenting the process.
- Students will be able to present their habitat to a partner using CS vocabulary.
- Students will be able to complete a self-evaluation rubric on 4 criteria.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF K-1.CT.4, K-1.CT.6, K-1.CT.8, K-1.CT.9, K-1.CT.10, K-1.NSD.1, K-1.CY.1, K-1.CY.2, and K-1.DL.2 simultaneously. CSTA 1A-AP-08, 1A-AP-10, 1A-AP-11, and 1A-AP-14 are all assessed through the project and rubric.

5. Assessment

Formative Assessment

Formative checkpoints: plan review at 15 min, circulating support during coding, and observation of partner presentations. Self-evaluation provides metacognitive data. SWD receive 1:1 check-ins. ELL students present with visual aids and bilingual vocabulary.

Summative Assessment

This IS the Unit A summative. 4-criteria rubric (Appendix A): Algorithm Design, Use of MakeCode Blocks, Debugging Evidence, Communication. Each on a 4-level scale. SWD accommodations: extended time, simplified rubric, reduced steps, 1:1 support. ELL accommodations: bilingual vocabulary, visual rubric, combined English/home language presentation.

6. Community Knowledge and Experience

The animal habitat theme connects to students' love of animals and science learning. Students choose their own animal — a pet, zoo favorite, or animal from cultural heritage. ELL students demonstrate sophisticated planning visually. SWD have multiple entry points and choose complexity.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	'Today is your big project! Build an animal habitat in Minecraft. Plan first, code second, present to a partner.'	Listen. Choose an animal. Think about habitat needs.
5–15 min	Distribute planning sheets. Model a quick example. Circulate and review.	Complete planning sheet with 5+ steps. Get teacher approval.
15–35 min	'Follow your plan. Debug as needed. Record bugs on your Debugging Log.' Circulate. Photograph screens.	Code habitat in MakeCode. Build using sequences and loops. Debug and document. Revise and retest.
35–42 min	Pair students. 'Show your habitat. Explain your code. Point out a bug you fixed.'	Present habitat to partner. Explain code. Show debugging evidence.
42–45 min	Distribute self-evaluation rubric. Collect all materials. Celebrate.	Complete self-evaluation. Turn in all materials. Celebrate.

Backup Plan: Paper-based: full planning and debugging on paper. Habitat drawn on poster paper with pseudocode. Partner presentations use posters. Rubric applied to paper project.

8. Differentiated Instruction

UDL Representation: verbal overview, visual example, written planning sheet. Action/Expression: planning, coding, debugging log, presenting. Engagement: free animal choice, authentic assessment. IEP: extended time, reduced steps (3+), simplified habitat, 1:1 support, visual rubric. ELL: bilingual vocabulary, Immersive Reader, sentence frames, combined language presentation. Extension: add data collection (count habitat blocks and create bar chart); write a paragraph for a class book.

9. Resources

Materials: projector, Minecraft Education, MakeCode, planning sheets, Debugging Log, self-evaluation rubric, sentence frames, all prior vocabulary cards. Assistive technology: Immersive Reader, adaptive inputs, large-print rubric.

10. Applications, Connections & Extensions

Final lesson of Unit A. Students in Unit B build on all foundational skills. Real-world: professional programmers plan, code, debug, and present daily. Extension: 'What I Learned in CS' mini-book; family showcase event.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Unit B: Coding FUNdamentals — Lesson Plans (3–5)

Block 1: Animal Conservation (Lessons B-1 through B-6)

Lesson B-1: Computational Thinking with Sea Turtles — Decomposition

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Computational Thinking with Sea Turtles — Decomposition
Duration	45 minutes

1. Content Area

This lesson introduces decomposition — breaking complex problems into smaller, manageable subproblems — within a sea turtle conservation context. Cross-curricular connections include science (sea turtle biology, conservation, migration) and ELA (expository writing about algorithms).

2. Purposes/Goals

Students will learn that decomposition is the foundation of computational thinking. The Big Idea is: Decomposing a complex problem into smaller subproblems is the foundation of computational thinking. Real-world problems are rarely simple; by breaking them apart, students learn to approach complexity with confidence.

3. Objectives

- Students will be able to apply decomposition to identify at least 3 subproblems within a complex challenge.
- Students will be able to write a 5+ step algorithm to solve one identified subproblem.
- Students will be able to compare their solution with a peer's and discuss tradeoffs.
- Students will be able to explain how decomposition helps manage complexity in a written exit ticket.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 2-3.CT.4 (Decompose a problem into smaller sub-problems) and 2-3.CT.6 (Compare and refine solutions). 2-3.CT.10 (Collaborate to solve a problem) is addressed through partner comparison. CSTA 1B-AP-08 (Compare and refine solutions) and 1B-AP-10 (Create programs using sequences, events, loops, and conditionals) are practiced. CSTA 1B-AP-11 (Decompose problems into subproblems) is the central standard.

5. Assessment

Formative Assessment

Teacher checks decomposition diagrams for at least 3 subproblems. MakeCode screenshots verify algorithm execution. The exit ticket (describe algorithm in 3 sentences) is scored for accuracy and use of vocabulary. SWD receive graphic organizers. ELL students use sentence frames.

Summative Assessment

This lesson launches the Unit B assessment arc. Decomposition is assessed in the summative rubric's 'Decomposition and Planning' criterion. This lesson teaches the foundational skill.

6. Community Knowledge and Experience

The sea turtle theme connects to students' environmental awareness and love of animals. Students share knowledge about conservation. ELL students may know about sea turtles from their home countries' coastlines. SWD participate through multi-modal activities. The teacher models curiosity about real-world conservation technology.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	Show a 2-minute video clip on sea turtle threats (plastic, predators, habitat loss). Ask: 'How could technology help these turtles?'	Watch video. Share ideas about technology and conservation.
5–8 min	'Let's decompose this problem. What are ALL the smaller problems inside "help the turtle"?' Chart student responses: clear the path, remove trash, protect eggs, track migration.	Brainstorm subproblems. Add ideas to class chart.
8–15 min	Model decomposition diagram: big problem at top, subproblems below. Students create their own.	Draw decomposition diagram with at least 3 subproblems.
15–35 min	Load Coding Fundamentals Block 1 Lesson 1 from the Library. Students code the Agent to clear the turtle's path. Circulate asking: 'Which subproblem are you solving right now?'	Open Minecraft. Code Agent to clear the turtle's path using MakeCode sequences. Test and debug.
35–42 min	Partner comparison: 'Did you write the same algorithm? What's different? Which approach has tradeoffs?' Guide discussion.	Compare algorithms with partner. Discuss tradeoffs. Identify similarities and differences.
42–45 min	Exit ticket: describe your algorithm in 3 sentences. Collect.	Write 3-sentence algorithm description. Submit.

Backup Plan: CS Unplugged 'Algorithms Sorting' activity: students decompose a sorting problem using physical cards offline.

8. Differentiated Instruction

UDL Representation: video, decomposition diagram, coding. Action/Expression: diagramming, coding, writing, verbal comparison. Engagement: real-world conservation context. IEP: pre-started decomposition with 2 subproblems filled in; simplified coding challenge; 1:1 support. ELL: bilingual vocabulary (decomposition/descomposición), visual diagram, sentence frames, partner support. Extension: solve TWO subproblems and connect the solutions.

9. Resources

Materials: projector, Minecraft Education, Coding Fundamentals Block 1 world, MakeCode, decomposition diagram worksheets, exit ticket forms, video clip (pre-loaded), vocabulary cards. Assistive technology: Immersive Reader, closed captions on video, adaptive inputs.

10. Applications, Connections & Extensions

Lesson B-2 builds directly on decomposition by adding loops. Real-world: engineers decompose building design; doctors decompose diagnosis. Extension: research one sea turtle conservation technology and present to the class.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson B-2: Loops for Polar Bears

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Loops for Polar Bears
Duration	45 minutes

1. Content Area

This lesson extends loop concepts to the 3–5 level, introducing iteration efficiency and the comparison between counted loops and conceptual while-loops. The context is polar bear habitat construction. Cross-curricular: science (Arctic ecosystems, climate change) and mathematics (efficiency, counting).

2. Purposes/Goals

Students will deepen their understanding of loops as tools for efficiency. The Big Idea is: Loops make programs efficient; we use them whenever we need to repeat a pattern. Students transition from simply using loops to analyzing WHEN and WHY loops are the appropriate choice.

3. Objectives

- Students will be able to identify inefficiency in code that uses 10+ repeated blocks and refactor it using a loop.
- Students will be able to explain the difference between 'repeat N times' and the concept of 'repeat while condition' in their own words.
- Students will be able to write a loop-based program that builds an Arctic shelter for a polar bear.
- Students will be able to compare two code samples and write which is better and why.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 2-3.CT.8 (Identify repetitive patterns in an algorithm) and 4-6.CT.8 (Develop programs using sequences, events, loops, and conditionals). CSTA 1B-AP-10 (Create programs with sequences, events, loops, and conditionals) and 1B-AP-11 (Decompose problems) are practiced.

5. Assessment

Formative Assessment

Teacher captures screenshots of refactored code. Written comparison (which code is better and why) is the primary artifact. SWD receive side-by-side visual comparison templates. ELL students use sentence frames for the written comparison.

Summative Assessment

Loop efficiency contributes to the Block 1 summative (B-6) and the capstone (B-18). The rubric's 'CS Concepts Applied' criterion requires demonstration of loops.

6. Community Knowledge and Experience

The polar bear context connects to students' awareness of climate change and Arctic ecosystems. Students share what they know about polar bears. ELL students may have seen polar bear media in their home countries. The teacher connects: 'Just as polar bears adapt to survive, programmers adapt code to be more efficient.'

7. Procedures

Time	Teacher will:	Student will:
0–5 min	Project a MakeCode program with 10 individual 'place block' commands. 'What's wrong with this code? It works, but...' Guide students to notice redundancy.	Examine the code. Identify that 10 separate commands are repetitive. Suggest using a loop.
5–15 min	Model refactoring: replace 10 commands with 'repeat 10 times: place block.' Run both. Ask: 'What if I need 100? Which code is easier to change?' Preview 'while' concept: 'What if I don't know how many times?'	Watch refactoring. Compare outcomes. Discuss: loop is easier to modify. Begin to understand 'repeat while' concept.
15–35 min	Load Arctic shelter Minecraft world. Students code an Agent to build an igloo-style shelter using loops. Circulate: 'How many times does your loop repeat? Could you use fewer blocks?'	Code loop-based shelter program in MakeCode. Build the shelter in Minecraft. Test and optimize loop count.
35–42 min	Display two student solutions side by side. 'Which is more efficient? Why?'	Compare solutions. Discuss efficiency. Identify which uses fewer blocks of code.
42–45 min	Exit ticket: 'Which code is better and why?' (two code samples given). Collect.	Write a comparison of the two code samples. Submit.

Backup Plan: Offline activity: students write out 20 identical instructions, then rewrite using 'repeat 20 times' notation. Compare writing time and paper used.

8. Differentiated Instruction

UDL Representation: projected code comparison, verbal modeling, hands-on coding. Action/Expression: coding, writing comparisons, verbal discussion. Engagement: Arctic conservation theme, efficiency competition. IEP: code samples pre-printed for comparison; reduced shelter complexity; 1:1 support. ELL: visual side-by-side comparison; sentence frames ('Code A is better because ____'); bilingual vocabulary. Extension: implement a 'while' conceptual loop using conditional blocks.

9. Resources

Materials: projector, Minecraft Education, Arctic shelter world, MakeCode, exit ticket with code samples, vocabulary cards. Assistive technology: Immersive Reader, adaptive inputs.

10. Applications, Connections & Extensions

Lesson B-3 adds debugging to looped code. Real-world: factory automation uses loops. Extension: research how many fish a polar bear eats daily and write a loop to simulate feeding.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson B-3: Debugging in the Wild — Panda Rescue

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Debugging in the Wild — Panda Rescue
Duration	45 minutes

1. Content Area

This lesson elevates debugging to a systematic, professional practice using a 7-step strategy. The panda rescue context provides engaging, complex debugging challenges. Cross-curricular: science (panda conservation, bamboo ecosystems) and ELA (procedural writing for debugging logs).

2. Purposes/Goals

Students will approach debugging as an iterative, professional practice. The Big Idea is: Testing and debugging are iterative, professional programming practices. At the 3–5 level, debugging moves beyond 'find the mistake' to systematic analysis, documentation, and retesting.

3. Objectives

- Students will be able to follow a 7-step debugging strategy: Read, Predict, Run, Compare, Identify, Fix, Retest.
- Students will be able to debug at least two errors in provided code with increasing complexity.
- Students will be able to complete a debugging log documenting what was wrong, how they fixed it, and what they tested.
- Students will be able to explain the difference between a syntax error and a logic error.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 2-3.CT.9 (Test and debug to refine a program) and 4-6.CT.9 (Test and debug programs systematically). CSTA 1B-AP-15 (Test and debug a program or algorithm to ensure it runs as intended) and 1B-CS-03 (Determine problems in the way computing devices function) are practiced.

5. Assessment

Formative Assessment

Teacher reviews debugging logs for completeness (what, how, tested). Observation during debugging checks systematic strategy use. SWD receive pre-structured logs. ELL students use sentence frames in their logs.

Summative Assessment

Debugging is assessed in the summative rubric's 'Debugging and Iteration' criterion. This lesson teaches the advanced strategy expected at the 3–5 level.

6. Community Knowledge and Experience

The panda rescue context engages students' empathy and environmental awareness. Students share prior knowledge about pandas. The growth-mindset framing continues from Unit A. ELL students benefit from the visual nature of code debugging.

7. Procedures

Time	Teacher will:	Student will:
0–3 min	Show a picture of tangled headphone cords. 'This is a lot like buggy code — tangled and confusing. But we can untangle it systematically.'	Make the connection between physical tangles and code bugs.
3–8 min	Post the 7-step strategy. Walk through an example: a panda's path-clearing code that has a wrong direction AND a wrong loop count (two bugs!).	Follow along with the 7-step process. Identify both bugs in the example code.
8–15 min	Model completing a debugging log on the projector: 'Bug 1: wrong direction on line 3. Fix: changed left to right. Retest: Agent now turns correctly but overshoots — Bug 2!'	Watch the modeling. Understand that debugging is often iterative — fixing one bug may reveal another.
15–35 min	Load Panda Rescue world with 4 pre-broken programs (increasing difficulty). Students debug each using the 7-step strategy and complete a log for each. Circulate: 'Which step are you on? What did you predict?'	Open each buggy program. Follow 7-step process. Complete debugging log for each bug found. Test fixes and document retesting.
35–42 min	Partner share: 'Compare logs. Did you find the same bugs? Did you fix them the same way?'	Compare debugging logs with partner. Discuss alternative fixes.
42–45 min	Exit ticket: debugging log (what was wrong, how fixed, what I tested). Collect.	Complete and submit debugging log exit ticket.

Backup Plan: Offline: students debug paper-based algorithms by tracing through step-by-step and marking errors with sticky notes.

8. Differentiated Instruction

UDL Representation: visual (tangled cords analogy), modeling (projected log), hands-on coding. Action/Expression: coding, writing logs, verbal comparison. Engagement: rescue narrative, systematic strategy provides structure. IEP: 2 programs instead of 4; pre-structured log; 1:1 support. ELL: sentence frames for logs; bilingual vocabulary; Immersive Reader. Extension: create a program with hidden bugs for a partner to debug; categorize bugs as syntax vs. logic.

9. Resources

Materials: projector, Minecraft Education, 4 pre-broken Panda Rescue programs, MakeCode, debugging log worksheets, 7-step strategy poster, vocabulary cards. Assistive technology: Immersive Reader, adaptive inputs.

10. Applications, Connections & Extensions

Lesson B-4 introduces pair programming, where debugging happens collaboratively. Real-world: QA testing teams in software companies. Extension: 'Bug Bounty' challenge — find the most bugs in a complex program.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson B-4: Pair Programming — Gray Wolf Collaboration

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Pair Programming — Gray Wolf Collaboration
Duration	45 minutes

1. Content Area

This lesson introduces the pair programming methodology — a professional software development practice where two programmers work together at one computer. The gray wolf pack context models collaborative problem-solving. Cross-curricular: social studies (teamwork, community roles) and ELA (active listening, constructive feedback).

2. Purposes/Goals

Students will experience collaborative programming and understand that diverse perspectives strengthen code. The Big Idea is: Programs built collaboratively are stronger; different perspectives reveal different errors and generate creative solutions. Pair programming is used at major technology companies and teaches essential communication skills.

3. Objectives

- Students will be able to explain the roles of Driver (typing) and Navigator (directing) in pair programming.
- Students will be able to rotate roles every 15 minutes and maintain productivity during transitions.
- Students will be able to use structured discussion stems ('I noticed... I wonder... What if we...') during collaboration.
- Students will be able to reflect on how collaboration changed their approach compared to working alone.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 2-3.CT.10 (Collaborate with peers to solve a computational problem) as the central standard. CSTA 1B-AP-16 (Incorporate existing code, media, and libraries into original programs, with attribution) introduces the concept of building on others' work. CSTA 1B-AP-11 (Decompose problems into subproblems) is practiced collaboratively.

5. Assessment

Formative Assessment

Teacher observes pair dynamics using a collaboration checklist (role adherence, respectful communication, productivity). The debrief written reflection is the primary artifact. SWD have simplified reflection prompts. ELL students use sentence stems.

Summative Assessment

Collaboration is assessed in the summative rubric's 'Collaboration and Roles' criterion. This lesson explicitly teaches the practices that criterion evaluates.

6. Community Knowledge and Experience

Wolf packs model natural collaboration — each wolf has a role. Students share examples of teamwork from sports, family, or cultural practices. ELL students may have strong collaborative traditions from their home cultures. SWD are paired intentionally for supportive partnerships.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	Show image of a gray wolf pack. 'Wolves work together — each has a role. Today, you'll code like a wolf pack. One person is the Driver (types), one is the Navigator (directs). You'll switch every 15 minutes.'	Learn about Driver and Navigator roles. Form pairs.
5–10 min	Model pair programming with a student volunteer. Demonstrate discussion stems: 'I noticed... I wonder... What if we...' Post stems on the board.	Watch demonstration. Practice discussion stems with partner.
10–25 min	Pairs work on Gray Wolf Habitat challenge in Minecraft. Driver types; Navigator reads the plan and directs. Circulate checking role adherence and communication.	Navigator directs; Driver types. Use discussion stems. Build wolf habitat collaboratively.
25–27 min	'Switch! Drivers become Navigators, Navigators become Drivers.'	Switch roles. Continue building from where partner left off.
27–40 min	Continue supporting pairs. Ask: 'What did the Navigator notice that the Driver missed?' Encourage constructive feedback.	Continue pair programming in new roles. Provide feedback using stems.
40–45 min	Debrief: 'How was coding with a partner different from coding alone? What did you learn from your partner?' Distribute reflection.	Write reflection using prompts: 'Working with a partner helped me _____ because _____. I learned that _____.'

Backup Plan: Offline pair programming: students use paper code cards. One person arranges cards (Driver) while the other directs and checks (Navigator).

8. Differentiated Instruction

UDL Representation: visual (wolf pack analogy), modeled demonstration, posted stems. Action/Expression: coding, discussion, written reflection. Engagement: social collaboration, role rotation. IEP: intentional pairing with supportive peer; simplified role expectations; visual role reminder card. ELL: discussion stems reduce language barrier; bilingual stems available; strategic pairing. Extension: triple programming with a 'Tester' role; reflect on which bugs the Navigator caught that the Driver missed.

9. Resources

Materials: projector, Minecraft Education, Wolf Habitat world, MakeCode, discussion stems poster, role cards (Driver/Navigator), reflection worksheets. Assistive technology: Immersive Reader, adaptive inputs.

10. Applications, Connections & Extensions

Lesson B-5 introduces variables, which pairs can use collaboratively. Real-world: pair programming at tech companies (Google, Microsoft). Extension: interview a family member about teamwork at their job.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson B-5: Variables — Animal Data Dashboard

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Variables — Animal Data Dashboard
Duration	45 minutes

1. Content Area

This lesson introduces variables as named containers that store changeable values. Students build a program tracking animal inventory using variables. Cross-curricular: mathematics (variables in algebra, graphing changing values) and science (data tracking, environmental monitoring).

2. Purposes/Goals

Students will understand that variables allow programs to store and update information dynamically. The Big Idea is: Variables are named containers that store values which can change while a program runs. Variables are the bridge between static sequences and dynamic, responsive programs.

3. Objectives

- Students will be able to define 'variable' as a named container that stores a value.
- Students will be able to create and use at least two variables in a MakeCode program (e.g., animalCount, foodSupply).
- Students will be able to modify variable values during program execution and observe the effect.
- Students will be able to graph variable values across multiple program runs to show change over time.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 2-3.CT.7 (Identify pieces of information that might change as a student completes a task) and 4-6.CT.7 (Identify pieces of information that can change and use variables to store and modify data). CSTA 1B-AP-09 (Create variables that represent different types of data) and 1B-DA-06 (Organize and present collected data visually) are practiced. CSTA 1B-DA-07 (Use data to highlight or propose cause-and-effect relationships) is introduced.

5. Assessment

Formative Assessment

Teacher checks variable naming conventions and correct assignment during coding. The graphed variable data is the primary formative artifact. SWD receive pre-labeled variable tracking sheets. ELL students use visual 'chest with label' analogy.

Summative Assessment

Variables are assessed in the summative's 'CS Concepts Applied' criterion. Students must demonstrate variable use to meet the standard.

6. Community Knowledge and Experience

Students connect variables to changing things in their lives: their age, the weather, the score in a game. The 'labeled chest' analogy makes the abstract concept concrete. ELL students relate to labeled containers from cooking and organizing. SWD benefit from the physical analogy.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	Hold up a labeled box: 'Animal Count.' Put 3 toy animals inside. 'This box is a variable. It has a name and a value. The value can change.' Remove 1 animal. 'Now what's the value?'	Watch demonstration. Answer: 'The value is 2.' Connect physical container to variable concept.
5–15 min	Open MakeCode. Create a variable called 'animalCount,' set to 0. Add code: when Agent collects an animal, increment animalCount by 1. Show the variable displayed on screen. 'Watch the number change!'	Watch. Open MakeCode. Create variables: animalCount and foodSupply.
15–30 min	Load Animal Dashboard world. Students code a program that tracks how many animals the Agent has helped and how much food has been distributed. Circulate: 'What's your animalCount now? What if you add 5 more?'	Code the dashboard program. Use variables to track animal and food counts. Run multiple times. Observe variables changing.
30–40 min	Distribute graphing worksheets. 'Run your program 5 times. After each run, record the variable values. Then graph them.' Model plotting one point.	Run program 5 times. Record variable values. Create a line graph showing how values change over runs.
40–45 min	'What pattern do you see in your graph? What would happen if you ran it 10 more times?' Exit ticket: 'A variable is ____ and it's useful because ____.'	Analyze graph. Make prediction. Complete exit ticket.

Backup Plan: Offline: labeled envelopes as variables. Students write a number on a card, put it in the envelope, then swap the card to show the variable changing. Graph envelope contents over time.

8. Differentiated Instruction

UDL Representation: physical labeled box, digital MakeCode variable, graphed data. Action/Expression: coding, graphing, writing, verbal. Engagement: animal tracking provides purpose; graphing connects CS to math. IEP: fewer variables (1 instead of 2); pre-labeled tracking sheet; 1:1 support. ELL: 'labeled chest' visual; bilingual vocabulary; sentence frames. Extension: create a variable that calculates a running total or average; design a dashboard with 3+ variables.

9. Resources

Materials: labeled box with toy animals, projector, Minecraft Education, Animal Dashboard world, MakeCode, graphing worksheets, colored pencils, vocabulary cards. Assistive technology: Immersive Reader, adaptive inputs.

10. Applications, Connections & Extensions

Lesson B-6 is the Block 1 summative where variables, loops, and debugging are assessed together. Real-world: variables in video game scores, weather apps, fitness trackers. Extension: design a 'variable tracker' for a real-life variable (daily temperature, steps walked).

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson B-6: Block 1 Summative — Animal Challenge Showcase

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Block 1 Summative — Animal Challenge Showcase
Duration	45 minutes

1. Content Area

This formative summative assesses Block 1 skills: decomposition, loops, variables, debugging, and collaboration. Students demonstrate learning through a structured showcase. Cross-curricular: ELA (presentation skills) and science (animal conservation themes).

2. Purposes/Goals

Students will demonstrate integrated mastery of Block 1 CS concepts. The Big Idea is: We can combine decomposition, loops, variables, and debugging to solve complex problems and present our solutions professionally. This mid-unit assessment provides data for instruction differentiation in Blocks 2 and 3.

3. Objectives

- Students will be able to create a Minecraft project that demonstrates loops, variables, and debugging.
- Students will be able to present their project to a partner explaining each CS concept used.
- Students will be able to complete a debugging log with at least 2 documented iterations.
- Students will be able to provide constructive feedback to a peer using structured stems.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 2-3.CT.4, 2-3.CT.6, 2-3.CT.7, 2-3.CT.8, 2-3.CT.9, and 2-3.CT.10. CSTA 1B-AP-08, 1B-AP-09, 1B-AP-10, 1B-AP-11, 1B-AP-15, and 1B-AP-16 are all assessed.

5. Assessment

Formative Assessment

Block 1 rubric scores on: loop usage, variable implementation, debugging log, and partner presentation. This is both formative (informing Block 2 instruction) and summative for Block 1. SWD accommodations: extended time, reduced project scope, simplified rubric. ELL accommodations: bilingual vocabulary, visual rubric, combined language presentation.

Summative Assessment

This IS the Block 1 summative assessment. Students are evaluated on 4 criteria: loop usage, variable implementation, debugging log completeness, and partner presentation quality.

6. Community Knowledge and Experience

Students choose their own animal challenge, drawing on personal interests and prior lessons. The showcase format celebrates all work. ELL students demonstrate through code and visuals. SWD choose appropriate complexity.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	Overview: 'Today you'll create a project showing loops, variables, and debugging. Plan for 5 minutes, code for 25 minutes, present for 10 minutes.'	Listen. Begin planning their animal challenge project.
5–10 min	Distribute planning sheets and debugging logs. Circulate and review plans.	Complete planning sheet. Identify which CS concepts they'll demonstrate.
10–35 min	Students code independently. Circulate providing support. 'Show me your variable. Where's your loop? Document your debugging.'	Code project using loops and variables. Debug and document iterations. Build in Minecraft.
35–42 min	Pair students for showcase. 'Present your project. Explain your loop, variable, and debugging process.' Provide feedback stems.	Present to partner. Explain CS concepts. Receive and give feedback.
42–45 min	Collect all materials. Celebrate Block 1 completion. Preview Block 2 (Space!).	Submit all materials. Celebrate.

Backup Plan: Paper-based showcase: planning sheet, pseudocode, hand-drawn Minecraft world, and written explanations of each CS concept.

8. Differentiated Instruction

UDL Representation: verbal overview, written planning sheet, visual Minecraft output. Action/Expression: planning, coding, debugging, presenting. Engagement: choice, authentic assessment, celebration. IEP: extended time, reduced scope, 1:1 support, visual rubric. ELL: bilingual vocabulary, sentence frames, combined language presentation. Extension: add a second animal to the project; create a tutorial for a younger student.

9. Resources

Materials: projector, Minecraft Education, MakeCode, planning sheets, debugging logs, feedback stems poster, rubric (printed). Assistive technology: Immersive Reader, adaptive inputs.

10. Applications, Connections & Extensions

Block 2 (Space) begins with B-7. Real-world: software demos and presentations are standard in the tech industry. Extension: write a blog post about the project for the class website.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Block 2: Space Exploration (Lessons B-7 through B-12)

Lesson B-7: Conditionals — Moon Landing

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Conditionals — Moon Landing
Duration	45 minutes

1. Content Area

This lesson introduces conditionals (if-then-else) and Boolean logic through a Moon Landing simulation. Students program the Agent to make decisions based on terrain conditions. Cross-curricular: science (space exploration, Moon geology) and mathematics (Boolean logic, true/false).

2. Purposes/Goals

Students will learn that conditionals enable programs to make decisions. The Big Idea is: Conditionals allow programs to respond differently to different situations, making them intelligent and adaptive. Programs without conditionals are rigid; with them, code becomes flexible.

3. Objectives

- Students will be able to define 'conditional' as a decision point where the program checks a condition and chooses a path.
- Students will be able to write an if-then-else statement in MakeCode that responds to at least two different terrain conditions.
- Students will be able to evaluate Boolean expressions (true/false) and predict program behavior.
- Students will be able to trace through a conditional program and explain each decision point.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 4-6.CT.8 (Develop programs using conditionals) as the primary standard. 2-3.CT.6 (Compare and refine solutions) is practiced through conditional optimization. CSTA 1B-AP-10 (Create programs with conditionals) is the central CSTA standard.

5. Assessment

Formative Assessment

Teacher checks conditional code for correct syntax and logic. Program trace worksheet is the formative artifact. SWD receive pre-written condition blocks to arrange. ELL students use visual flowcharts.

Summative Assessment

Conditionals are required in the capstone's 'CS Concepts Applied' criterion. This lesson introduces the concept; subsequent lessons combine it with loops and variables.

6. Community Knowledge and Experience

Decision-making connects to everyday choices: 'If it's raining, bring an umbrella; else, wear sunscreen.' Students share daily conditional decisions. The Moon Landing context sparks curiosity about space exploration. ELL students connect through universal interest in space.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	'What do you do if it's cold outside? What if it's warm?' Write on board as if/else: 'IF cold THEN wear jacket ELSE wear t-shirt.' 'This is a conditional — a decision!'	Share daily decisions. See them written as if/else. Understand the conditional structure.
5–15 min	Open MakeCode. Model an if-then-else block: 'IF Agent detects crater THEN jump ELSE walk forward.' Run and demonstrate both outcomes.	Watch both outcomes. Open MakeCode. Begin experimenting with if-then-else blocks.
15–35 min	Load Moon Landing world. Students program Agent to navigate lunar terrain: IF flat ground THEN move forward; IF crater THEN jump; IF rock THEN turn. Circulate: 'What conditions are you checking?'	Code conditional navigation program. Test on different terrain sections. Add more conditions as they encounter obstacles.
35–42 min	Distribute program trace worksheet. Students trace through their code step by step, writing the condition and outcome at each decision point.	Complete program trace. Write: 'At step 3, the condition was ____ so the program did ____.'
42–45 min	Exit ticket: 'A conditional is _____. It makes programs better because _____. Collect.	Complete exit ticket.

Backup Plan: Offline: 'Human Robot' conditional activity. One student is the robot; another gives conditional commands: 'IF you see a red card, clap. ELSE, stomp.' Teacher holds up cards.

8. Differentiated Instruction

UDL Representation: real-life analogy, MakeCode visual blocks, program trace. Action/Expression: coding, tracing, writing. Engagement: space exploration theme. IEP: fewer conditions (1 if-then instead of if-then-else); pre-built condition blocks to arrange; 1:1 support. ELL: visual flowchart showing condition paths; bilingual vocabulary; sentence frames. Extension: nested conditionals (if inside if); create a decision tree diagram for the lunar surface.

9. Resources

Materials: projector, Minecraft Education, Moon Landing world, MakeCode, program trace worksheets, exit tickets, vocabulary cards. Assistive technology: Immersive Reader, adaptive inputs.

10. Applications, Connections & Extensions

Lesson B-8 adds operators for more complex conditions. Real-world: conditionals in traffic lights, ATMs, smart home devices. Extension: design a 'conditional adventure story' where the reader makes choices.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson B-8: Operators — Mercury Math

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Operators — Mercury Math
Duration	45 minutes

1. Content Area

This lesson introduces comparison and arithmetic operators within a Mercury exploration context. Students use operators to create more complex conditional expressions and mathematical calculations in their programs. Cross-curricular: mathematics (operations, comparison, expressions) and science (Mercury planet facts, temperature extremes).

2. Purposes/Goals

Students will use operators to make programs calculate and compare. The Big Idea is: Operators enable programs to perform calculations and make comparisons, creating smarter and more dynamic code. Operators transform programs from simple command sequences into tools that can reason about data.

3. Objectives

- Students will be able to use arithmetic operators (+, -, ×, ÷) in MakeCode to calculate values.
- Students will be able to use comparison operators (>, <, =, ≠) in conditional expressions.
- Students will be able to combine operators with variables to create dynamic calculations.
- Students will be able to predict the output of an expression involving multiple operators.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 4-6.CT.4 (Decompose a problem and create a solution using algorithms involving variables and operators) and 4-6.CT.7 (Use variables to store and modify data). CSTA 1B-AP-09 (Create variables) and 1B-AP-10 (Create programs with conditionals) are extended with operator usage.

5. Assessment

Formative Assessment

Teacher reviews programs for correct operator usage. Expression evaluation worksheet is scored for accuracy. SWD receive simplified expressions with single operators. ELL students use visual operator reference cards.

Summative Assessment

Operators contribute to the summative's 'CS Concepts Applied' criterion by enabling more sophisticated conditional logic and variable manipulation.

6. Community Knowledge and Experience

Mercury Math connects programming to familiar arithmetic while making it exciting through space exploration. Students share their math experiences. The extreme temperatures on Mercury provide dramatic comparison data. ELL students often excel in mathematics-based activities.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	Mercury facts: 'Mercury's day side is 430°C, night side is -180°C. How could a program know which side is which? It needs to COMPARE temperatures!' Introduce comparison operators.	Listen to Mercury facts. Recognize the need for comparison: 'IF temperature > 400 THEN it's the day side.'
5–15 min	Model arithmetic and comparison operators in MakeCode. Create variables: dayTemp = 430, nightTemp = -180, difference = dayTemp - nightTemp. Use IF difference > 500 THEN display warning.	Watch modeling. Open MakeCode. Create variables and operators.
15–35 min	Load Mercury Math world. Students program a temperature monitoring system: calculate averages, compare to thresholds, trigger alerts. Circulate: 'What operator are you using? What does it calculate?'	Code temperature monitoring program. Use arithmetic operators for calculations. Use comparison operators in conditionals.
35–42 min	Expression evaluation worksheet: predict the output of 5 expressions before running them.	Evaluate expressions on paper. Predict outputs. Then run in MakeCode to check.
42–45 min	Review answers together. Exit ticket: write one expression using a variable and an operator, and predict the result. Collect.	Complete and submit exit ticket.

Backup Plan: Offline: 'Human Calculator' game — students are variables holding number cards. Teacher calls out operations; students calculate and update their cards.

8. Differentiated Instruction

UDL Representation: real-world context, visual operator reference, hands-on coding. Action/Expression: coding, written evaluation, verbal. Engagement: space and math integration. IEP: single-operator expressions; visual operator chart; 1:1 support. ELL: universal math symbols reduce language barrier; visual reference cards; sentence frames. Extension: create a program that uses nested operators; design a unit converter (Celsius to Fahrenheit).

9. Resources

Materials: projector, Minecraft Education, Mercury Math world, MakeCode, expression worksheets, operator reference cards, vocabulary cards. Assistive technology: Immersive Reader, adaptive inputs.

10. Applications, Connections & Extensions

Lesson B-9 introduces events. Real-world: spreadsheet formulas, calculator apps, weather dashboards use operators. Extension: create a 'Space Math Quiz' program using operators and conditionals.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson B-9: Events — Venus Launch Sequence

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Events — Venus Launch Sequence
Duration	45 minutes

1. Content Area

This lesson introduces event-driven programming through a Venus rocket launch simulation. Students learn that events are triggers that cause code to execute. Cross-curricular: science (Venus atmosphere, rocket science) and ELA (cause-and-effect reasoning).

2. Purposes/Goals

Students will understand that events allow programs to respond to user actions and system triggers. The Big Idea is: Events trigger code execution, making programs interactive and responsive to the world around them. Event-driven programming is the paradigm behind every interactive application students use.

3. Objectives

- Students will be able to define 'event' as a trigger that causes specific code to run.
- Students will be able to create at least two different event handlers in MakeCode (e.g., on chat command, on Agent collision).
- Students will be able to design a launch sequence with events occurring in the correct order.
- Students will be able to explain the difference between sequential code and event-driven code.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 4-6.CT.8 (Develop programs using events) as a primary standard. 2-3.CT.6 (Compare and refine solutions) is practiced. CSTA 1B-AP-10 (Create programs with events) is the central CSTA standard.

5. Assessment

Formative Assessment

Teacher checks for correct event handler syntax and at least two different event types. The launch sequence documentation is the formative artifact. SWD receive pre-built event blocks to connect. ELL students use visual event-trigger diagrams.

Summative Assessment

Events are assessed in the capstone's 'CS Concepts Applied' criterion. This lesson provides the foundational understanding.

6. Community Knowledge and Experience

Launch countdowns are exciting and universally understood. Students share experiences with interactive technology (pressing buttons, voice commands, motion sensors). ELL students connect through the universal excitement of space launches.

7. Procedures

Time	Teacher will:	Student will:
0–3 min	'3... 2... 1... LAUNCH!' Ask: 'What just happened? A specific THING (the countdown reaching zero) caused a specific ACTION (the launch). In programming, that's called an event.'	Experience the countdown excitement. Connect the trigger (reaching zero) to the action (launch).
3–10 min	Open MakeCode. Show event blocks: 'on chat command "launch"', 'on player walk.' Demonstrate: when I type "launch," this code runs. 'The chat command is the EVENT that triggers the code.'	Watch. Understand that events wait for triggers before running code.
10–15 min	Model a 3-stage launch sequence: Event 1 (chat command "fuel"): fill tanks. Event 2 (chat command "ignite"): start engines. Event 3 (chat command "launch"): lift off.	Watch the 3-stage sequence. Identify each event and its triggered code.
15–35 min	Load Venus Launch Sequence world. Students code their own multi-event launch sequence. Must include at least 2 different event types. Circulate: 'What event triggers this code? What happens if the event doesn't occur?'	Code launch sequence with multiple event handlers. Test by triggering events in different orders. Observe that order matters for events.
35–42 min	'How is event-driven code different from the sequential code we wrote before?' Facilitate discussion. Students write a comparison.	Discuss differences. Write: 'Sequential code runs in order. Event-driven code waits for triggers.'
42–45 min	Exit ticket: 'An event is _____. My launch sequence used events such as _____. Collect.	Complete and submit.

Backup Plan: Offline: 'Event Cards' game. Students hold trigger cards (clap, whistle, bell) and action cards (jump, spin, sit). When the teacher activates a trigger, students with matching action cards perform them.

8. Differentiated Instruction

UDL Representation: kinesthetic countdown, visual event blocks, hands-on coding. Action/Expression: coding, writing, discussion. Engagement: space launch excitement. IEP: 1 event type instead of 2; pre-built event blocks; 1:1 support. ELL: visual event-trigger diagram; bilingual vocabulary; sentence frames. Extension: create a program with 5+ events; add sound effects to each event stage.

9. Resources

Materials: projector, Minecraft Education, Venus Launch Sequence world, MakeCode, event documentation worksheets, vocabulary cards. Assistive technology: Immersive Reader, adaptive inputs.

10. Applications, Connections & Extensions

Lesson B-10 combines loops with conditionals. Real-world: smartphone apps (tap to open, shake to undo), video games, smart home devices. Extension: design an event map for a familiar app.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson B-10: Loops + Conditionals — Mars Exploration

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Loops + Conditionals — Mars Exploration
Duration	45 minutes

1. Content Area

This lesson combines loops and conditionals into integrated programs for Mars surface exploration. Students create programs that repeat actions while making decisions within each iteration. Cross-curricular: science (Mars geology, rover exploration) and mathematics (iterative problem-solving).

2. Purposes/Goals

Students will combine loops and conditionals to create adaptive, repeating programs. The Big Idea is: Combining loops and conditionals creates programs that can repeat actions while making intelligent decisions each time. This integration represents a significant leap in programming sophistication.

3. Objectives

- Students will be able to write a program that uses a loop containing a conditional statement.
- Students will be able to trace through a loop-conditional program and predict the output for different inputs.
- Students will be able to explain why combining loops and conditionals is more powerful than using either alone.
- Students will be able to design a Mars rover navigation algorithm that adapts to terrain.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 4-6.CT.8 (Develop programs using loops and conditionals) as the combined standard. 4-6.CT.6 (Compare solutions based on efficiency) is introduced. CSTA 1B-AP-10 (Create programs with loops and conditionals) is practiced at a combined level.

5. Assessment

Formative Assessment

Teacher reviews combined loop-conditional code for correct nesting. The program trace with prediction is scored for accuracy. SWD receive simplified terrain (2 conditions instead of 4). ELL students use flowcharts.

Summative Assessment

Combined structures are essential for the capstone's 'CS Concepts Applied' criterion at the 'Meets' or 'Exceeds' level.

6. Community Knowledge and Experience

Mars rovers (Curiosity, Perseverance) are exciting real-world examples of combined loops and conditionals. Students share what they know about Mars exploration. The connection to real NASA programs validates programming as a real career.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	Show Mars rover image. 'This rover drives across Mars repeating the same process: check terrain, decide action, move, repeat. That's a loop with conditionals inside!'	Connect rover operation to loop + conditional concept.
5–15 min	Model in MakeCode: 'repeat 10 times { IF obstacle detected THEN turn ELSE move forward }.' Run. 'Watch how the Agent adapts each time through the loop.'	Watch combined loop-conditional program. Understand that the conditional runs INSIDE each loop iteration.
15–35 min	Load Mars Exploration world with varied terrain. Students code a rover navigation algorithm: repeat while moving, check for rocks (turn), craters (jump), flat (move). Circulate: 'How many times does your loop run? What conditions are you checking?'	Code adaptive rover program. Test on Mars terrain. Debug when the rover gets stuck. Add more conditions for different obstacles.
35–42 min	Program trace: given a specific terrain map, trace through the loop-conditional code and predict the Agent's path.	Complete program trace on paper. Mark each decision and the resulting action.
42–45 min	Exit ticket: 'Why is combining loops and conditionals more powerful than using either alone?' Collect.	Write explanation. Submit.

Backup Plan: Offline: 'Mars Terrain Walk.' Students walk a grid. At each square, flip a card (rock/crater/flat) and follow their pre-written conditional rules. Count how many times the loop runs.

8. Differentiated Instruction

UDL Representation: rover image, visual code modeling, hands-on coding. Action/Expression: coding, tracing, writing. Engagement: NASA/Mars context. IEP: fewer conditions (2 instead of 4); pre-built loop structure; 1:1 support. ELL: flowchart showing loop-conditional path; bilingual vocabulary; sentence frames. Extension: add a variable to count obstacles encountered; optimize code for fewest turns.

9. Resources

Materials: projector, Minecraft Education, Mars Exploration world, MakeCode, program trace worksheets, vocabulary cards. Assistive technology: Immersive Reader, adaptive inputs.

10. Applications, Connections & Extensions

Lesson B-11 introduces networks and cybersecurity. Real-world: self-driving cars use loops with conditionals constantly. Extension: research a real Mars rover mission and map its decision algorithm.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson B-11: Networks — Jupiter Station

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Networks — Jupiter Station
Duration	45 minutes

1. Content Area

This lesson introduces computer networks, data packets, and cybersecurity through a Jupiter space station communication simulation. Students learn how computers send and receive information. Cross-curricular: science (Jupiter, radio waves in space) and social studies (communication systems, privacy).

2. Purposes/Goals

Students will understand how computers communicate over networks and why security matters. The Big Idea is: Computers communicate by sending data in packets over networks, and cybersecurity protects that data. Understanding networks demystifies the internet and empowers students as informed digital citizens.

3. Objectives

- Students will be able to explain how data travels in packets across a network.
- Students will be able to describe the function of at least two network components (router, server, client).
- Students will be able to identify at least two cybersecurity threats and corresponding protections.
- Students will be able to simulate packet transmission in a Minecraft redstone communication system.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 4-6.NSD.2 (Explain how data is sent and received over physical or wireless paths) and 4-6.NSD.4 (Describe how information is broken down and sent in packets through multiple devices). 4-6.CY.1 (Determine the types of personal information that should remain private) and 4-6.CY.2 (Identify the threats to safe and secure use of technology) are addressed. CSTA 1B-NI-04 (Model how information is broken down into smaller pieces, transmitted, then reassembled) and 1B-NI-05 (Discuss real-world cybersecurity problems) are practiced.

5. Assessment

Formative Assessment

Teacher checks packet simulation for accurate message reconstruction. The cybersecurity threat/protection matching is the formative artifact. SWD receive pre-assembled packet cards. ELL students use visual network diagrams.

Summative Assessment

Network and cybersecurity concepts contribute to the capstone's 'CS Concepts Applied' criterion and prepare students for responsible technology use throughout their lives.

6. Community Knowledge and Experience

Students connect networks to their daily internet use — streaming, messaging, gaming. The Jupiter Station context makes abstract networking concepts tangible. ELL students relate to communication across distances (connecting with family in other countries). SWD benefit from the physical packet-passing activity.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	'How does a message get from your phone to your friend's phone?' Chart student theories. Introduce: 'Data travels in packets — small pieces of information that travel through networks.'	Share theories about how messages travel. Learn the word 'packet.'
5–10 min	Unplugged: write a sentence on a card, tear it into 3 pieces (packets), number them. Pass pieces through 3 'routers' (students). Receiver reassembles. 'That's how networks work!'	Participate in packet-passing activity. Reassemble the message from numbered pieces.
10–20 min	Show network diagram: client → router → server → router → client. Introduce cybersecurity: 'What if someone intercepts a packet? That's a cybersecurity threat. Encryption scrambles the message so only the right person can read it.'	Study network diagram. Understand the flow. Learn about encryption as protection.
20–35 min	Load Jupiter Station world. Students build a Minecraft redstone communication system that sends a signal (packet) from one station to another. Circulate: 'What happens if your signal is interrupted? How could you protect it?'	Build redstone communication system. Send and receive signals. Troubleshoot interrupted signals.
35–42 min	Cybersecurity matching: distribute cards with threats (hacking, phishing, malware) and protections (passwords, encryption, firewalls). Students match threats to protections.	Match threat cards to protection cards. Discuss with partner.
42–45 min	Exit ticket: 'Data travels in ____ over _____. One cybersecurity threat is _____ and we protect against it with _____.' Collect.	Complete and submit exit ticket.

Backup Plan: Full offline version: packet-passing relay plus paper network diagram labeling. Cybersecurity discussion and matching remain the same.

8. Differentiated Instruction

UDL Representation: physical packet passing, visual diagram, Minecraft redstone. Action/Expression: physical activity, building, matching, writing. Engagement: space station context, hands-on redstone. IEP: fewer threats/protections to match; pre-assembled redstone circuit; 1:1 support. ELL: visual network diagram; bilingual vocabulary; picture-based matching cards. Extension: design an encryption system (simple Caesar cipher); research a real cybersecurity incident.

9. Resources

Materials: projector, Minecraft Education, Jupiter Station world, message cards, threat/protection matching cards, network diagram poster, vocabulary cards. Assistive technology: Immersive Reader, adaptive inputs.

10. Applications, Connections & Extensions

Lesson B-12 is the Block 2 summative. Real-world: Wi-Fi, Bluetooth, cellular networks, internet security. Extension: trace the path of a real email from sender to receiver.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson B-12: Neptune Expedition — Block 2 Summative

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Neptune Expedition — Block 2 Summative
Duration	45 minutes

1. Content Area

This summative assesses Block 2 skills: conditionals, operators, events, combined loops/conditionals, and networks. Students design a Neptune deep-space expedition incorporating all concepts. Cross-curricular: science (Neptune, deep space) and ELA (presentation, technical writing).

2. Purposes/Goals

Students will demonstrate integrated mastery of Block 2 concepts through an expedition design. The Big Idea is: Complex programs integrate multiple CS concepts — conditionals, operators, events, loops, and awareness of networks — to solve sophisticated problems. This assessment marks students' growth from basic to intermediate programming.

3. Objectives

- Students will be able to design and code a Neptune expedition program using conditionals, loops, variables, operators, and events.
- Students will be able to document their development process including planning, debugging, and iteration.
- Students will be able to present their expedition to peers explaining each CS concept demonstrated.
- Students will be able to provide constructive peer feedback.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 4-6.CT.4, 4-6.CT.6, 4-6.CT.7, 4-6.CT.8, 4-6.CT.9, 4-6.NSD.2, and 4-6.CY.2. CSTA 1B-AP-08, 1B-AP-09, 1B-AP-10, 1B-AP-11, 1B-AP-15, 1B-NI-04, and 1B-NI-05 are assessed.

5. Assessment

Formative Assessment

Block 2 rubric assesses: conditionals usage, operator integration, event implementation, debugging documentation, and presentation quality. SWD: extended time, reduced concept requirements (4 of 6), 1:1 support. ELL: bilingual vocabulary, visual rubric, combined language presentation.

Summative Assessment

This IS the Block 2 summative. Students demonstrate integration of all Block 2 concepts in a single project.

6. Community Knowledge and Experience

Neptune is mysterious and distant — students connect to the excitement of exploring the unknown. The open-ended expedition design allows personal creativity. All students choose their own expedition focus.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	Overview: 'Your Neptune Expedition must include conditionals, operators, events, a loop with a conditional, and awareness of how data would travel back to Earth. Plan, code, present.'	Listen. Begin planning expedition.
5–10 min	Distribute planning sheets. Circulate and review.	Complete planning sheet identifying each CS concept they'll demonstrate.
10–35 min	Students code independently. Circulate providing support. 'Show me your conditional. Where's your event? How do operators help?'	Code Neptune expedition. Demonstrate all required concepts. Debug and document.
35–42 min	Partner presentations with feedback using 'I like / I wonder / What if' stems.	Present to partner. Give and receive feedback.
42–45 min	Collect materials. Celebrate Block 2. Preview Block 3 (Time Travel!).	Submit materials. Celebrate.

Backup Plan: Paper-based: planning, pseudocode, drawn world, and written explanations of each concept.

8. Differentiated Instruction

UDL Representation: verbal overview, written planning, visual output. Action/Expression: planning, coding, presenting. Engagement: space exploration, choice. IEP: reduced concepts (4/6), extended time, 1:1 support. ELL: bilingual vocabulary, visual rubric, combined language presentation. Extension: add network security features to the expedition; create a mission control dashboard.

9. Resources

Materials: projector, Minecraft Education, MakeCode, planning sheets, debugging logs, rubric, feedback stems. Assistive technology: Immersive Reader.

10. Applications, Connections & Extensions

Block 3 (Time Travel) begins with B-13. Real-world: complex software projects integrate all these concepts. Extension: write a mission report for the Neptune expedition.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Block 3: Time Travel (Lessons B-13 through B-18)

Lesson B-13: Dynamic Variables — Dinosaur Age

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Dynamic Variables — Dinosaur Age
Duration	45 minutes

1. Content Area

This lesson deepens variable understanding with scope, increment/decrement operations, and dynamic updating in a Dinosaur Age context. Cross-curricular: science (paleontology, dinosaur eras) and mathematics (number operations, patterns).

2. Purposes/Goals

Students will master dynamic variable manipulation. The Big Idea is: Variables can be dynamically updated during program execution using increment and decrement operations, enabling programs to track changing data in real time. This elevates variables from simple storage to dynamic program control.

3. Objectives

- Students will be able to increment and decrement variables using += and -= operations.
- Students will be able to create a program that tracks and updates multiple variables simultaneously.
- Students will be able to explain variable scope (where a variable is accessible) in simple terms.
- Students will be able to design a dinosaur tracking system that dynamically updates population counts.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 4-6.CT.7 (Use variables to store and modify data) at an advanced level. 4-6.CT.4 (Decompose a problem using variables and operators) is practiced. CSTA 1B-AP-09 (Create variables that represent different types of data) is extended.

5. Assessment

Formative Assessment

Teacher reviews variable update operations for correctness. The dinosaur tracking output is the formative artifact. SWD use pre-built variable templates. ELL students use visual counter diagrams.

Summative Assessment

Advanced variable skills are essential for the capstone's 'CS Concepts Applied' criterion at the 'Exceeds' level.

6. Community Knowledge and Experience

Dinosaurs are universally engaging for elementary students. Students share dinosaur knowledge and connect counting to real paleontological data collection. ELL students connect through visual, universal excitement about dinosaurs.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	'Imagine you're a paleontologist tracking dinosaur populations. Every time you discover a new T-Rex, you add 1 to your count. That's incrementing a variable!' Demonstrate with physical counter.	Watch physical counter. Understand increment as adding to a variable.
5–15 min	In MakeCode: create dinoCount = 0. Code: when Agent finds a dinosaur, dinoCount += 1. When a dinosaur disappears, dinoCount -= 1. Show the variable changing in real time.	Watch. Open MakeCode. Create dinoCount and foodSupply variables with increment/decrement.
15–35 min	Load Dinosaur Age world. Students build a tracking system: count different dinosaur types, track food supply depletion, trigger alerts when population falls below threshold. Circulate: 'What does your variable equal now? What happens when you decrement?'	Code tracking system. Use += and -= operations. Watch variables change. Trigger conditional alerts based on variable thresholds.
35–42 min	Discussion: 'What happened to your variables over time? Did any reach zero? What would happen if the variable was only accessible in one part of the code?' Introduce scope concept simply.	Discuss variable changes. Begin understanding that where you create a variable matters (scope).
42–45 min	Exit ticket: 'If dinoCount starts at 10 and I run += 3 then -= 5, what is dinoCount? Explain.' Collect.	Calculate and explain. Submit.

Backup Plan: Offline: students use physical counters (abacus or number lines) to model increment/decrement. Track dinosaur picture cards and update tallies.

8. Differentiated Instruction

UDL Representation: physical counter, visual MakeCode, hands-on coding. Action/Expression: coding, calculating, writing. Engagement: dinosaur theme. IEP: single variable, simplified operations, 1:1 support. ELL: visual counter diagrams, bilingual vocabulary. Extension: use variables to calculate ratios; implement a population growth model.

9. Resources

Materials: projector, Minecraft Education, Dinosaur Age world, MakeCode, physical counter, vocabulary cards. Assistive technology: Immersive Reader.

10. Applications, Connections & Extensions

Lesson B-14 introduces functions. Real-world: fitness trackers use incrementing variables. Extension: design a dinosaur population simulation over 10 time periods.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson B-14: Functions — Ancient Rome Aqueducts

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Functions — Ancient Rome Aqueducts
Duration	45 minutes

1. Content Area

This lesson introduces functions as reusable blocks of code that perform specific tasks. Students build an aqueduct system using modular functions. Cross-curricular: social studies (Ancient Roman engineering) and mathematics (modularity, efficiency).

2. Purposes/Goals

Students will learn that functions enable code reuse and organizational clarity. The Big Idea is: Functions enable code reuse and clarity by packaging instructions into named, reusable units. Functions are the foundation of modular programming and professional software development.

3. Objectives

- Students will be able to define 'function' as a named, reusable set of instructions.
- Students will be able to create at least two custom functions in MakeCode and call them from the main program.
- Students will be able to explain how functions reduce code repetition and improve readability.
- Students will be able to design an aqueduct system using modular function calls.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 4-6.CT.10 (Develop and document a program using an iterative design process involving planning, testing, debugging, and refining) through function-based modular design. 4-6.CT.6 (Compare solutions for effectiveness and efficiency) is practiced. CSTA 1B-AP-08 (Compare and refine multiple solutions) is used. CSTA 1B-AP-11 (Decompose problems) is extended to functional decomposition.

5. Assessment

Formative Assessment

Teacher checks for correct function definition and multiple function calls. The aqueduct design documentation is the formative artifact. SWD use pre-defined functions to call. ELL students use visual function diagrams showing input/output.

Summative Assessment

Functions are required in the capstone's 'CS Concepts Applied' criterion. This lesson provides the foundational skill.

6. Community Knowledge and Experience

Roman aqueducts are an engineering marvel that connects CS to history. Students share knowledge about ancient civilizations. The building analogy (modular construction) makes functions concrete. ELL students engage through visual building and universal fascination with ancient engineering.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	Show image of a Roman aqueduct. 'Romans built aqueducts from repeating sections — arches. Each arch is the same design, used over and over. In programming, that's a FUNCTION — a reusable design.'	Study aqueduct image. Recognize the repeating arch pattern. Connect to function concept.
5–15 min	In MakeCode: create a function called 'buildArch' that builds one arch. Then in the main program: call buildArch() 5 times. 'I wrote the arch code once and used it 5 times!'	Watch function creation and calling. Understand: write once, use many times.
15–35 min	Load Ancient Rome world. Students create functions: buildArch(), buildChannel(), buildSupport(). Call them to construct a complete aqueduct. Circulate: 'How many times do you call buildArch? What if you wanted to change the arch size — how many places would you edit?'	Define custom functions. Build aqueduct by calling functions. Appreciate that changing the function definition updates ALL calls.
35–42 min	Discussion: 'How is using functions different from copying and pasting the same code?' Students write a comparison.	Discuss functions vs. copy-paste. Write: 'Functions are better because ____.'
42–45 min	Exit ticket: 'A function is _____. Functions help programmers because ____.' Collect.	Complete and submit.

Backup Plan: Offline: LEGO function activity. Students define a 'function' as a specific LEGO build pattern, then call it by building that pattern multiple times from the same instruction card.

8. Differentiated Instruction

UDL Representation: visual aqueduct analogy, code modeling, hands-on building. Action/Expression: coding, writing, building. Engagement: ancient history context. IEP: use pre-defined functions; simplified aqueduct; 1:1 support. ELL: visual function diagrams; bilingual vocabulary. Extension: create functions with parameters (buildArch(height)); design a function library.

9. Resources

Materials: projector, Minecraft Education, Ancient Rome world, MakeCode, function documentation worksheets, aqueduct images, vocabulary cards. Assistive technology: Immersive Reader.

10. Applications, Connections & Extensions

Lesson B-15 introduces refactoring. Real-world: all professional software uses functions. Extension: research Roman engineering and create a digital presentation.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson B-15: Refactoring — Wild West Code Cleanup

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Refactoring — Wild West Code Cleanup
Duration	45 minutes

1. Content Area

This lesson teaches refactoring — improving existing code without changing its functionality. Students take messy, working programs and make them cleaner, more efficient, and better documented. Cross-curricular: ELA (editing and revision) and social studies (Wild West frontier resourcefulness).

2. Purposes/Goals

Students will learn that writing code is iterative — good programmers continuously improve their work. The Big Idea is: Refactoring improves code quality without changing what the code does, making it more efficient, readable, and maintainable. This mirrors the writing process of revision and editing.

3. Objectives

- Students will be able to define 'refactoring' as improving code without changing its behavior.
- Students will be able to identify at least three improvements in a provided program (redundant code, unclear naming, missing comments).
- Students will be able to refactor a program by consolidating repeated code into loops or functions.
- Students will be able to add meaningful comments to their code explaining what each section does.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 4-6.CT.10 (Develop and document a program using an iterative design process) as the primary standard. 4-6.CT.6 (Compare solutions for effectiveness and efficiency) is directly practiced. CSTA 1B-AP-15 (Test and debug programs) is extended to include code quality improvement.

5. Assessment

Formative Assessment

Teacher compares before and after code for measurable improvement (fewer blocks, better names, added comments). The refactoring log is the formative artifact. SWD refactor 1 program instead of 2. ELL students use a visual checklist.

Summative Assessment

Refactoring skill is assessed in the capstone through the 'Debugging and Iteration' criterion, which includes code quality improvement.

6. Community Knowledge and Experience

The Wild West 'cleanup' metaphor connects refactoring to cleaning up a town. Students relate to the editing process in writing. ELL students benefit from the visual comparison of before/after code. SWD appreciate the systematic checklist approach.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	Show two writing samples: messy first draft vs. clean final draft. 'Writing gets better with revision. Code gets better with refactoring. Same idea!'	Compare writing samples. Connect revision to refactoring.
5–15 min	Show a messy but working MakeCode program (20 blocks with repetition, unclear variable names, no comments). Ask: 'It works, but what's wrong with it?' List issues. Model refactoring: replace repeated code with a loop, rename variables clearly, add comments.	Identify issues in the messy code. Watch refactoring process. See the improvement.
15–35 min	Load Wild West Code Cleanup world with 2 provided programs. Students refactor each using a checklist: (1) Eliminate repeated code, (2) Use meaningful names, (3) Add comments, (4) Test that it still works. Circulate: 'Did your refactored code produce the same output?'	Open each program. Follow the refactoring checklist. Consolidate code, rename variables, add comments. Test to ensure same behavior.
35–42 min	Before/after showcase: students display original and refactored code side by side. 'How many blocks did you save? What's clearer now?'	Compare before and after. Count blocks saved. Discuss improvements with partner.
42–45 min	Exit ticket: 'Refactoring is _____. One way I improved my code was _____.' Collect.	Complete and submit.

Backup Plan: Offline: students receive a messy handwritten set of instructions (e.g., for building a LEGO model) and rewrite them more clearly and efficiently.

8. Differentiated Instruction

UDL Representation: writing analogy, visual code comparison, checklist. Action/Expression: coding, writing logs, verbal. Engagement: Wild West cleanup theme, before/after satisfaction. IEP: 1 program, simplified checklist, 1:1 support. ELL: visual checklist, bilingual vocabulary, partner support. Extension: refactor their own earlier projects; write a 'Code Quality Guide' for classmates.

9. Resources

Materials: projector, Minecraft Education, 2 provided programs, MakeCode, refactoring checklist, vocabulary cards. Assistive technology: Immersive Reader.

10. Applications, Connections & Extensions

Lesson B-16 introduces multiplayer collaboration. Real-world: professional code reviews. Extension: peer code review — refactor a partner's program and explain improvements.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson B-16: Future City — Multiplayer Collaboration

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Future City — Multiplayer Collaboration
Duration	45 minutes

1. Content Area

This lesson uses Minecraft’s multiplayer features for a collaborative city-building project emphasizing division of labor, integration testing, and teamwork. Cross-curricular: social studies (urban planning, community design) and ELA (collaborative writing, communication).

2. Purposes/Goals

Students will collaborate to build a complex system from individually coded components. The Big Idea is: Complex programs are built by teams; each person contributes components that must integrate into a working whole. Division of labor and integration are core principles of professional software development.

3. Objectives

- Students will be able to divide a complex project into components assigned to different team members.
- Students will be able to code their assigned component using appropriate CS concepts.
- Students will be able to integrate their component with teammates’ components and test the combined system.
- Students will be able to resolve integration issues through collaborative debugging.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 4-6.CT.10 (Develop programs using an iterative, collaborative design process) and 2-3.CT.10 (Collaborate to solve problems). CSTA 1B-AP-16 (Incorporate existing code and collaborate) and 1B-AP-11 (Decompose problems) are practiced in a team context.

5. Assessment

Formative Assessment

Teacher assesses individual contributions through documented component plans and collaborative debugging through integration logs. SWD have clearly defined roles with appropriate complexity. ELL students contribute through coding while bilingual peers support communication.

Summative Assessment

Collaboration and integration skills are assessed in the capstone's 'Collaboration and Roles' criterion.

6. Community Knowledge and Experience

City building connects to students' communities. Teams discuss: 'What does our city need?' Drawing on diverse perspectives, including ELL students who may envision features from their home countries. SWD contribute meaningfully within their role.

7. Procedures

Time	Teacher will:	Student will:
0–5 min	'Today we build a Future City as a team. Each person codes one building or system. Then we connect them. Just like real software teams!' Assign groups of 3–4.	Form groups. Listen to the team project overview.
5–15 min	Groups plan: divide the city into components (power system, transportation, housing, park). Each member claims a component and writes a plan. Teacher approves plans.	Discuss as a group. Divide components. Each member writes a plan for their component.
15–35 min	Students code individually on their component, then join the multiplayer Minecraft world to integrate. Circulate: 'Does your component work with your teammate's? What needs to change?'	Code individual component. Join multiplayer world. Place component in the city. Test integration. Debug connection issues with teammates.
35–42 min	Teams present their cities. Each member explains their component. 'How did you resolve integration issues?'	Present as a team. Each member explains their contribution.
42–45 min	Reflection: 'What was hard about working as a team? What would you do differently?' Collect.	Write reflection. Submit.

Backup Plan: Offline: paper-based city design. Each student designs one building on a separate sheet; the team assembles them on a large poster and identifies integration issues (roads don't connect, buildings overlap).

8. Differentiated Instruction

UDL Representation: verbal overview, visual city plan, hands-on building. Action/Expression: planning, coding, building, presenting. Engagement: collaborative creation, community relevance. IEP: clearly defined role with appropriate scope; 1:1 support; simplified component. ELL: bilingual planning sheet; coding tasks minimize language barriers; supportive team placement. Extension: add a network infrastructure component; write a city planning proposal.

9. Resources

Materials: projector, Minecraft Education multiplayer, MakeCode, component planning sheets, integration logs, vocabulary cards. Assistive technology: Immersive Reader.

10. Applications, Connections & Extensions

Lesson B-17 explores real-world CS impacts. Real-world: software development teams at major companies. Extension: research how a real city uses technology and present findings.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson B-17: CS in the Real World — Impacts of Computing

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	CS in the Real World — Impacts of Computing
Duration	45 minutes

1. Content Area

This lesson explores the societal impacts of computing technology, including both benefits and challenges. Students examine accessibility, digital equity, and responsible innovation. Cross-curricular: social studies (citizenship, social justice) and ELA (argumentative writing, media literacy).

2. Purposes/Goals

Students will analyze how computing technology affects society and develop a critical perspective on innovation. The Big Idea is: Computing technology shapes and is shaped by society; technologists have a responsibility to create inclusive, equitable solutions. This lesson moves beyond coding skills to computational citizenship.

3. Objectives

- Students will be able to identify at least three positive impacts of computing on society.
- Students will be able to identify at least two challenges or risks posed by computing technology.
- Students will be able to explain what accessibility means in technology design and why it matters.
- Students will be able to propose one way technology could be designed more inclusively.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 4-6.IC.1 (Describe how computing technology has changed the world) and 4-6.IC.6 (Identify and discuss how computing technologies could improve or be designed to improve accessibility). CSTA 1B-IC-18 (Discuss computing technologies that have changed the world) and 1B-IC-19 (Describe ways in which computing practices might improve accessibility) are practiced.

5. Assessment

Formative Assessment

Teacher assesses discussion contributions and the written proposal for inclusivity. The proposal is the formative artifact. SWD share perspectives from personal experience with accessibility. ELL students use visual and verbal modes.

Summative Assessment

Impacts of computing are assessed in the capstone's 'Presentation and Reflection' criterion, where students must reflect on the broader significance of their project.

6. Community Knowledge and Experience

Students with disabilities bring invaluable perspectives on accessibility. ELL students experience the digital divide directly and can speak to equity issues. The teacher validates all perspectives and ensures that computing impacts are discussed across cultures and communities.

7. Procedures

Time	Teacher will:	Student will:
0–10 min	Show 3 images: a student using assistive technology, a doctor using telehealth, a family video-calling overseas. 'Computing has changed all their lives. How?' Then show 2 more: cyberbullying scenario, e-waste. 'But it's not all positive.'	Discuss each image. Identify positive impacts and challenges.
10–20 min	Focus on accessibility: 'What does it mean for technology to be accessible? Who might be left out if we don't think about accessibility?' Show Minecraft's Immersive Reader as an accessibility feature. 'This was designed so everyone can play.'	Discuss accessibility. Identify who benefits from accessible design. Recognize Immersive Reader as an example.
20–35 min	Groups of 3: research one computing technology (assigned: social media, AI, video games, wearables, internet). Fill out a T-chart: positive impacts / challenges. Groups share with class.	Research assigned technology. Complete T-chart. Present findings to class.
35–42 min	'Now, choose one challenge your group identified. Write a proposal for how technology could be designed more inclusively to address it.' Model a brief proposal.	Write an inclusivity proposal addressing one technology challenge. Describe the problem and propose a design solution.
42–45 min	Share 2–3 proposals. 'You are the next generation of technology designers. Remember: technology should work for everyone.' Collect.	Listen to proposals. Reflect on their role as future designers.

Backup Plan: Full offline lesson: same discussion, T-chart, and proposal without needing Minecraft. Use printed images and newspaper clippings.

8. Differentiated Instruction

UDL Representation: images, discussion, reading. Action/Expression: speaking, writing, drawing. Engagement: real-world relevance, personal connection to technology. IEP: share lived experience with accessibility; simplified proposal format; 1:1 support. ELL: image-based discussion; bilingual materials; group support. Extension: design an accessible Minecraft world; write a persuasive essay about digital equity.

9. Resources

Materials: projector, images (printed or projected), T-chart worksheets, proposal templates, vocabulary cards. Minecraft optional for Immersive Reader demonstration. Assistive technology: Immersive Reader demo.

10. Applications, Connections & Extensions

Lesson B-18 is the Grand Exhibition Capstone. Real-world: tech company ethics boards, accessibility guidelines (WCAG). Extension: interview a person with a disability about their technology use; design an accessibility improvement for the school website.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Lesson B-18: Grand Exhibition Capstone

Candidate	[Candidate Name]
Date	March 2026
Cooperating Teacher	[To be filled]
Grade Level	3–5
Subject Area	Computer Science / Digital Literacy
Title of Unit	Coding FUNdamentals
Lesson Title	Grand Exhibition Capstone
Duration	45 minutes

1. Content Area

This culminating lesson spans two 45-minute periods. Students design, code, and present a complete Minecraft world demonstrating all six core CS concepts (decomposition, loops, conditionals, variables, functions, and events). The Grand Exhibition is a gallery walk with self-evaluation and peer feedback. Cross-curricular: all subjects through student-chosen themes.

2. Purposes/Goals

Students will synthesize their complete CS learning into a single, original project and communicate their understanding to an audience. The Big Idea is: Programming is a creative, iterative process of problem-solving; complex programs are built from simple, reusable parts. This capstone celebrates students' growth from beginning coders to confident computational thinkers.

3. Objectives

- Students will be able to design and code a Minecraft world that demonstrates all 6 core CS concepts: decomposition, loops, conditionals, variables, functions, and events.
- Students will be able to document their development process including planning, debugging iterations, and refactoring decisions.
- Students will be able to present their project clearly, explaining each CS concept and design choice.
- Students will be able to complete a self-evaluation using the 5-criteria rubric.
- Students will be able to provide constructive peer feedback using 'I like / I wonder / What if' stems.

These objectives support IEP goal development by providing measurable, observable benchmarks adaptable to individual student plans. In a heterogeneous classroom, each objective can be scaffolded up or down to ensure all learners work toward meaningful, appropriate goals.

4. National and/or NYS Standards

This lesson addresses NYS CSDF 4-6.CT.1, 4-6.CT.4, 4-6.CT.6, 4-6.CT.7, 4-6.CT.8, 4-6.CT.9, 4-6.CT.10, 4-6.IC.1, and 4-6.IC.6 comprehensively. CSTA 1B-AP-08, 1B-AP-09, 1B-AP-10, 1B-AP-11, 1B-AP-15, 1B-AP-16, 1B-IC-18, and 1B-IC-19 are all assessed through the project and rubric.

5. Assessment

Formative Assessment

Day 1: Teacher reviews plans before coding begins. Circulates during development. Day 2: Observes gallery walk presentations. Self-evaluation and peer feedback forms provide metacognitive data. SWD: 1:1 check-ins at each transition, adapted rubric, extended time. ELL: bilingual vocabulary cards during presentations, visual rubric, combined language presentation accepted.

Summative Assessment

This IS the Unit B summative assessment. Students are evaluated using a 5-criteria rubric (Appendix B): Decomposition and Planning, CS Concepts Applied, Debugging and Iteration, Collaboration and Roles, Presentation and Reflection. Each on a 4-level scale. SWD accommodations: up to 3 class periods, simplified rubric, reduced concept requirements (4 of 6), 1:1 support, alternative presentation format. ELL accommodations: bilingual vocabulary, visual rubric, combined English/home language presentation, visual documentation of process.

6. Community Knowledge and Experience

The open-ended theme choice allows students to demonstrate CS skills through personally meaningful projects — a student might build a version of their neighborhood, recreate a historical event, or design a futuristic invention. This ensures cultural relevance and personal investment. ELL students often produce impressive visual work. SWD choose themes aligned with their strengths. The gallery walk format celebrates every student's contribution equally.

7. Procedures

Time	Teacher will:	Student will:
Day 1: 0–10 min	Overview: 'This is your Grand Exhibition. You'll design a complete Minecraft world showing ALL 6 CS concepts. Day 1: plan and code. Day 2: present at our gallery walk.' Distribute comprehensive planning packet.	Listen. Review planning packet sections: Theme, Decomposition (subproblems), Concepts Checklist, Code Plan, Debugging Log.
Day 1: 10–15 min	Students complete planning section. Teacher circulates reviewing plans. 'Do you have all 6 concepts planned? Which concept are you most excited about?'	Complete planning section. Check off each CS concept on the checklist. Get teacher approval.
Day 1: 15–43 min	Development time. Students code, build, debug, and iterate. Teacher circulates asking: 'Show me your function. Where's your conditional? Document that bug!' Photograph student screens for portfolio.	Code comprehensive project in MakeCode and Minecraft. Demonstrate all 6 concepts. Debug and document in the Debugging Log. Refactor for clarity.
Day 1: 43–45 min	Save work. 'Tomorrow is the gallery walk. Practice explaining your project — you'll need to show each CS concept to visitors.'	Save project. Begin mentally preparing presentation.

Day 2: 0–5 min	Set up gallery walk stations. Each student's project is displayed. Distribute peer feedback forms ('I like / I wonder / What if').	Set up their station. Prepare to present.
Day 2: 5–30 min	Gallery walk: half the class presents while the other half visits (15 min), then switch. Teacher visits each station asking: 'Tell me about your conditional. Why did you use a function here?'	Presenters: explain project to visitors, pointing out each CS concept. Visitors: observe, ask questions, complete feedback forms for each station visited.
Day 2: 30–40 min	Distribute self-evaluation rubric. Students read peer feedback and rate themselves on 5 criteria. Teacher collects all materials: plan, debugging log, peer feedback, self-evaluation.	Read peer feedback. Complete self-evaluation rubric (rate each of 5 criteria). Turn in all materials.
Day 2: 40–45 min	Celebration and reflection. 'Look at what you've accomplished — from your first sequence to a complete project with 6 CS concepts. You are computer scientists!' Final reflection: 'What are you most proud of? What will you remember?'	Celebrate. Write final reflection. Share one proud moment with the class.

Backup Plan: Paper-based capstone: comprehensive planning document, hand-drawn world design, pseudocode for all 6 concepts, written explanations, and poster presentation. Peer feedback and self-evaluation remain the same.

8. Differentiated Instruction

UDL Representation: verbal overview, comprehensive planning packet, visual checklist. Action/Expression: planning, coding, building, presenting, writing self-evaluation. Engagement: complete creative freedom; gallery walk audience; celebration. IEP: up to 3 periods; 4 of 6 concepts; pre-started planning template; 1:1 support; alternative presentation (seated, recorded, or 1:1 with teacher). ELL: bilingual vocabulary throughout; visual planning packet; code demonstrations supplement verbal explanations; combined language accepted; peer feedback in any language. Extension: mentor a younger student through their project; write a 'Lessons Learned' blog post; create a video tutorial of their favorite CS concept.

9. Resources

Materials: projector, Minecraft Education, MakeCode, comprehensive planning packets, debugging logs, peer feedback forms, self-evaluation rubrics (printed), celebration supplies. Assistive technology: Immersive Reader, adaptive inputs, amplification for presentation.

10. Applications, Connections & Extensions

This is the final lesson of Unit B. Students who continue studying CS will build on all concepts mastered here. Real-world: software engineers present at conferences; game designers showcase at exhibitions. The skills practiced — planning, coding, debugging, collaborating, presenting, and reflecting — are the exact skills used in professional CS careers. Extension: submit project to a school or district technology showcase; create a portfolio page documenting the project.

11. Personal Reflection

Complete the following prompts after teaching this lesson:

- What went well in this lesson? What evidence do you have?
- What would you change if you taught this lesson again? Why?
- What did you learn about the students from this lesson?
- What did you learn about yourself as an educator?
- How confident are you that you met the instructional, emotional, and social needs of all students, including English Language Learners and students with disabilities? What evidence supports your assessment?
- How did you construct a meaningful learning community during this lesson? What moves did you make to ensure every student felt valued and included?

Appendix A: Unit A Summative Rubric (Lesson A-10)

Animal Habitat Challenge — 4-Criteria Assessment Rubric

Criterion	4 — Exceeds	3 — Meets	2 — Approaching	1 — Beginning
Algorithm Design	5+ step plan on paper; code matches plan exactly; demonstrates planning sophistication	4-step plan; code mostly matches plan	Partial plan; code partially works	No prior plan; code does not complete task
Use of MakeCode Blocks	Uses sequence + loop + at least 1 additional block type creatively	Uses sequence + loop correctly	Uses sequence only	Uses fewer than 3 blocks total
Debugging Evidence	Fixed 2+ errors; each documented with what happened and why	Fixed at least 1 error and documented the process	Shows attempt to fix an error but no documentation	No debugging evidence
Communication	Clearly explains code to partner using CS vocabulary with confidence	Explains code with some CS vocabulary	Partial explanation; limited vocabulary	Unable to explain code

Appendix B: Unit B Summative Rubric (Lesson B-18)

Grand Exhibition Capstone — 5-Criteria Assessment Rubric

Criterion	4 — Exceeds	3 — Meets	2 — Approaching	1 — Beginning
Decomposition & Planning	Written plan with named subproblems; code matches plan; demonstrates planning sophistication	Written plan; code mostly matches	Partial plan; code partially works	No plan
CS Concepts Applied	All 6 concepts present + creative extension or optimization	All 6 concepts present and correctly implemented	4–5 concepts present	Fewer than 4 concepts
Debugging & Iteration	Detailed log; 3+ iterations documented with analysis	Log with 2 iterations documented	Some evidence of iteration	No debugging log
Collaboration & Roles	Active contributor; documented contributions; helped peers proactively	Participates actively in all roles	Participates with prompting	Limited participation
Presentation & Reflection	Clear explanation; rich self-reflection; constructive peer feedback given and received	Explanation + self-reflection present	Partial explanation or reflection	Unable to explain or reflect

Appendix C: Exit Ticket Templates

K–2 Exit Ticket Template

Name: _____ Date: _____

Today I learned: _____

I am still wondering: _____

Rate yourself: Not yet confident / Getting there / Got it!

3–5 Exit Ticket Template

Name: _____ Date: _____

The CS concept I practiced today was: _____

My code did _____ because _____

One thing I would change is: _____

Appendix D: Vocabulary Reference Sheets

K–2 Vocabulary (with Minecraft Associations)

Term	Definition + Minecraft Connection
Algorithm	A set of steps in a specific order to complete a task. Minecraft: The sequence of MakeCode blocks you arrange.
Sequence	Instructions that run one after another in order. Minecraft: A row of movement blocks.
Loop	An instruction to repeat steps a certain number of times. Minecraft: The 'repeat' block wrapping movement blocks.
Debug	Finding and fixing errors in code. Minecraft: When the Agent goes the wrong way and you fix the block.
Hardware	Physical parts of a computer you can touch. Minecraft: Like the keyboard you use to play.
Software	Instructions (programs) that tell hardware what to do. Minecraft: MakeCode is software.
Data	Information we collect and organize. Minecraft: Counting blocks in your world.
Input	Information or commands we give to a computer. Minecraft: Keyboard presses, mouse clicks.
Output	Information the computer shows us. Minecraft: What you see on screen.
Digital Citizen	Someone who uses technology safely and responsibly. Minecraft: Following safety rules online.

3–5 Vocabulary (with Code Examples)

Term	Definition + Minecraft Connection
Decomposition	Breaking a complex problem into smaller, manageable subproblems. Code: Dividing 'build a city' into buildHouse(), buildRoad(), buildPark().
Conditional	A decision point: IF condition THEN action ELSE alternative. Code: <pre>if (agentDetect('block')) { agent.turn('left') } else { agent.move('forward') }</pre>
Variable	A named container that stores a value which can change. Code: <pre>let score = 0; score += 10;</pre>
Function	A named, reusable set of instructions. Code: <pre>function buildArch() { /* arch code */ }</pre> — call it 5 times to build 5 arches.

Event	A trigger that causes specific code to run. Code: <code>player.onChat('launch', function() { /* launch code */ })</code>
Operator	A symbol that performs a calculation or comparison (+, -, >, <, ==). Code: <code>if (temperature > 100) { alert('Too hot!') }</code>
Iteration	Repeating a set of instructions using a loop. Code: <code>for (let i = 0; i < 10; i++) { agent.move('forward') }</code>
Debugging	Systematically finding and fixing errors in code. Process: Read → Predict → Run → Compare → Identify → Fix → Retest.
Modularity	Organizing code into independent, reusable sections (functions/modules). Principle: Each module does one thing well.
Network/Packet	Computers communicate by breaking data into small packets sent over networks. Analogy: A letter cut into numbered pieces, sent separately, reassembled by the receiver.

Appendix E: Teacher Quick Reference — Minecraft Education Setup

Account Requirements

Students need Microsoft 365 Education accounts (provided by the school district IT department). Teachers must have a Minecraft Education license assigned to their account. Accounts are managed through the Microsoft 365 Admin Center.

Sharing Worlds with Students

There are three methods for sharing Minecraft worlds with students: (1) Google Classroom link — paste the Minecraft world link into a Google Classroom assignment; (2) In-game join code — the teacher hosts the world and shares a join code displayed on screen; (3) Library search — students search the Minecraft Education Library by lesson name. Method 3 is recommended for Coding Fundamentals lessons as they are pre-loaded in the Library.

Device Capacity and Requirements

Chromebook	Up to 15 concurrent users; requires Minecraft Education for Chromebook app from Google Play Store
Windows PC	Up to 25 concurrent users; best performance; requires Minecraft Education desktop app
iPad	Up to 20 concurrent users; requires Minecraft Education app from App Store
Mac	Up to 20 concurrent users; requires Minecraft Education desktop app

Common Issues and Fixes

World won't load	Exit Minecraft completely, wait 30 seconds, and re-enter. If issue persists beyond 10 minutes, restart the device.
MakeCode editor not appearing	Press 'C' to open the Code Builder. If it still doesn't appear, check that Code Connection is installed.
Agent not responding	Ensure the Agent is spawned (type '/agent' in chat). Move the Agent to the player's location with 'agent tp.'
Students can't join multiplayer	Verify all students are on the same network. Check that the host has enabled multiplayer in world settings.
Performance lag	Reduce render distance in settings. Close other applications. Limit the number of active entities in the world.
Student lost their work	Check the 'My Worlds' section for auto-saved copies. Enable 'Export World' for manual backups before each session.

Game Modes

Creative Mode: Students can build freely, fly, and have unlimited blocks. Recommended for open-ended building lessons and the summative assessments. Adventure Mode: Students can interact with the world but cannot break or place blocks outside of their designated area. Recommended for guided lessons where the teacher has pre-built the world. Survival Mode: Students must gather resources and manage health. Not recommended for coding lessons as it distracts from CS objectives.

References

Bile, A. (2022). *Development of intellectual and scientific abilities through game-programming in Minecraft. Education and Information Technologies*. <https://doi.org/10.1007/s10639-022-10894-z>

Butler, D., Slattery, E. J., Marshall, K., & O'Leary, M. (2023). *Primary school students' experiences using Minecraft Education during a national project-based initiative: An Irish study*. *TechTrends*. <https://doi.org/10.1007/s11023-00851-z>

Computer Science Teachers Association. (2017). *CSTA K–12 Computer Science Standards (Revised)*. <https://csteachers.org/wp-content/uploads/2025/03/csta-k-12-computer-science-standards-revised.pdf>

Hébert, C., & Jenson, J. (2021). *Teaching with sandbox games: Minecraft, game-based learning, and 21st century competencies*. *Canadian Journal of Learning and Technology*, 47(1). <https://doi.org/10.21432/CJLT27990>

Microsoft Education. (2020). *Coding FUNdamentals: New computer science lessons for students of all ages*. <https://education.minecraft.net/en-us/blog/coding-fundamentals-new-computer-science-lessons-for-students-of-all-ages>

Microsoft Education. (2024). *Introduce coding in Minecraft Education*. Microsoft Learn. <https://learn.microsoft.com/en-us/training/modules/introduce-coding-minecraft-education-edition/>

New York State Education Department. (2020). *New York State K-12 Computer Science and Digital Fluency Learning Standards*. <https://www.nysed.gov/curriculum-instruction/computer-science-and-digital-fluency-learning-standards>

Wiggins, G., & McTighe, J. (2005). *Understanding by design* (2nd ed.). ASCD.